

2. ESP32开发基础

2. ESP32开发基础

2.1 ESP32开发环境配置

- 2.1.1 Arduino IDE安装方法
- 2.1.2 界面分布介绍
- 2.1.3 Arduino IDE界面设置
- 2.1.4 ESP32芯片包安装
- 2.1.5 库文件导入（必看！！）
- 2.1.6 Arduino程序下载

2.2 单个总线舵机控制

- 2.2.1 项目介绍
- 2.2.2 实验流程
- 2.2.3 模块说明
- 2.2.4 程序下载
- 2.2.5 运行效果
- 2.2.6 程序分析

2.3 总线舵机状态读取

- 2.3.1 项目介绍
- 2.3.2 实验流程
- 2.3.3 模块说明
- 2.3.4 程序下载
- 2.3.5 运行效果
- 2.3.6 程序分析

2.4 同步控制多个总线舵机

- 2.4.1 项目介绍
- 2.4.2 实验流程
- 2.4.3 模块说明
- 2.4.4 程序下载
- 2.4.5 运行效果
- 2.4.6 程序分析

2.5 按键扫描与蜂鸣器

- 2.5.1 项目介绍
- 2.5.2 实验流程
- 2.5.3 模块说明
- 2.5.4 程序下载
- 2.5.5 运行效果
- 2.5.6 程序分析

2.6 OLED显示

- 2.6.1 项目介绍
- 2.6.2 实验流程
- 2.6.3 模块说明
- 2.6.4 程序下载
- 2.6.5 运行效果
- 2.6.6 程序分析

2.7 FLASH文件系统操作

- 2.7.1 项目介绍
- 2.7.2 实验流程
- 2.7.3 模块说明
- 2.7.4 程序下载
- 2.7.5 运行效果

- 2.7.6 程序分析
- 2.8 WIFI编程操作
 - 2.8.1 项目介绍
 - 2.8.2 实验流程
 - 2.8.3 模块说明
 - 2.8.4 程序下载
 - 2.8.5 运行效果
 - 2.8.6 程序分析
- 2.9 ADC电压读取
 - 2.9.1 项目介绍
 - 2.9.2 实验流程
 - 2.9.3 模块说明
 - 2.9.4 程序下载
 - 2.9.5 运行效果
 - 2.9.6 程序分析

2.1 ESP32开发环境配置

2.1.1 Arduino IDE安装方法

Arduino IDE 是一款专门为 Arduino 单片机而设计的软件，功能强大。无论是哪个版本，其安装过程都是相同的，本节以 Arduino-2.2.1 的 windows 版软件为例进行讲解：

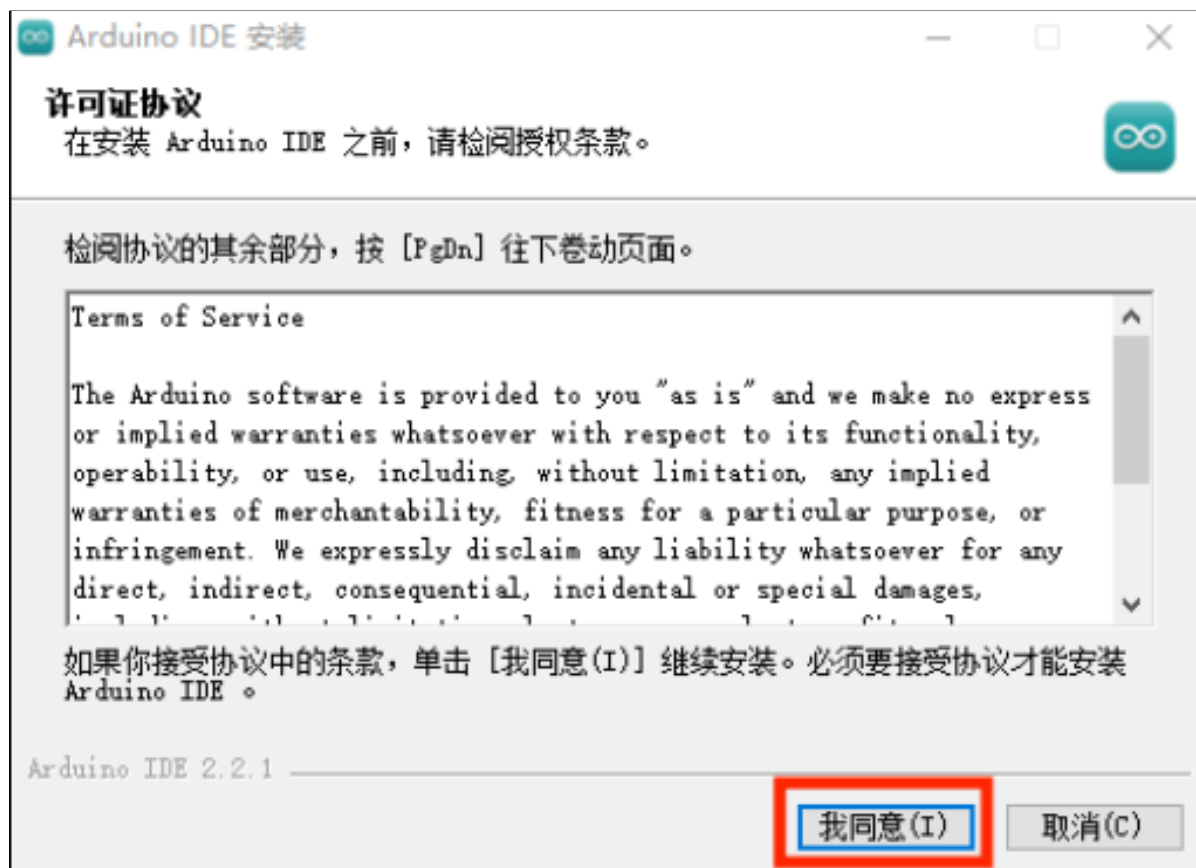
Note

Mac版安装方法可在本节目录中查看。

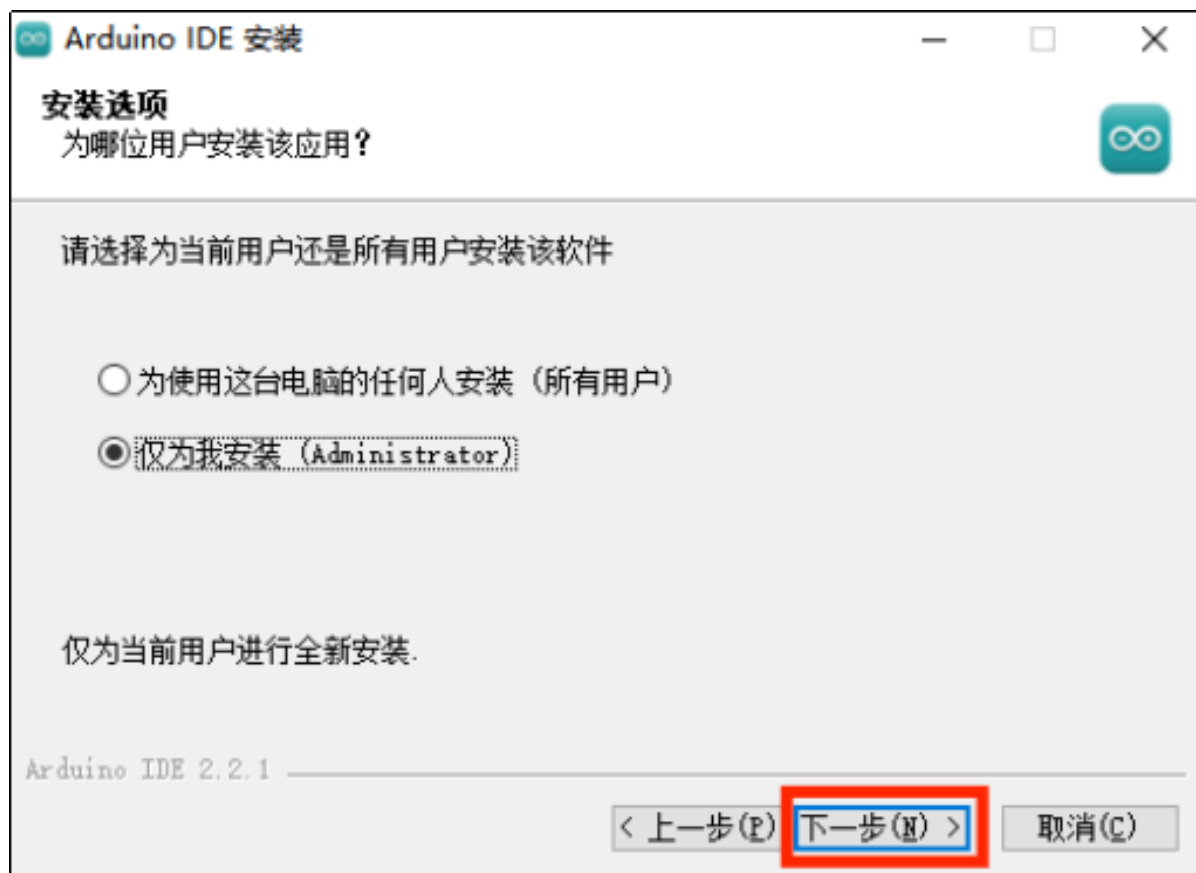
1. 在本文档同路径下找到“02 程序源码\01 Arduino安装包及开发包”文件夹里的**ArduinoIDE-2.2.1.exe**的安装包，如下所示，双击打开。（若想下载最新版软件，可通过 [Arduino 官网](#) 进行下载）。



2. 点击**"我同意"**，进入安装。

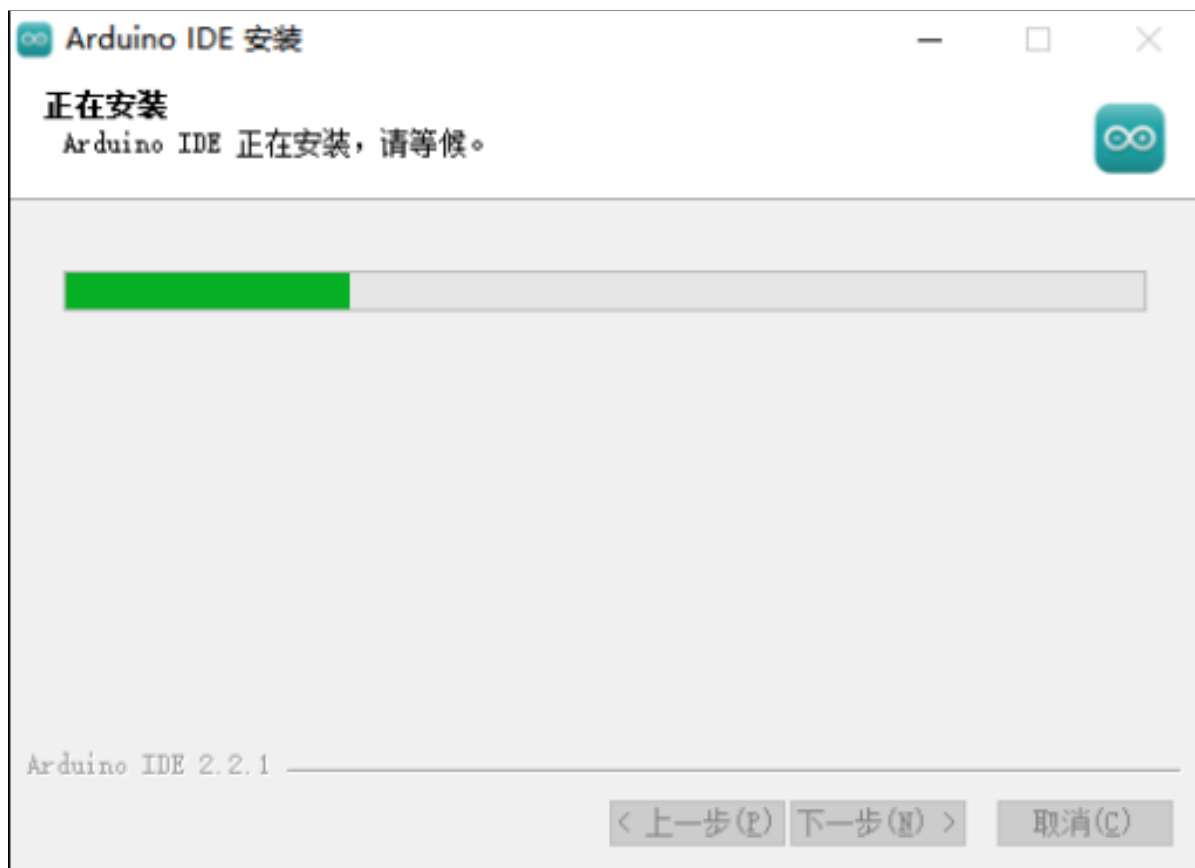


3. 选择默认勾选选项即可, 点击"下一步", 进入下一步操作。



4. 点击"浏览"选择安装的路径, 然后点击"安装"开始安装。

5. 等待软件安装完成。



Note

安装过程如果提示需要芯片驱动的安装, 请勾选"始终信任来自Arduino LLC的软件(A)", 然后点击"安装"即可。

6. 安装完成后, 点击"完成"。



2.1.2 界面分布介绍

Arduino IDE 的主界面如下图，可分为5个区域：



1. 菜单栏：负责Arduino IDE相关设置工作。

图标	功能
文件	可以新建或打开项目文件，也可以对界面进行首选项设置
编辑	编辑选项，可对代码进行注释、缩进、查找等文本编辑
项目	项目选项，对整个项目进行设置，编译运行、添加库文件等
工具	工具选项，可以选择开发板和端口以及获得开发板信息等
帮助(H)	帮助选项，帮助用户入门，常见问题解决等

2. 工具栏：项目相关的一些工具，包括编译程序、下载程序、串口监视器等工具。

图标	功能
	校验，验证一个程序是否编写无误，若无误则编译该项目
	下载，下载程序到 Arduino 控制器上

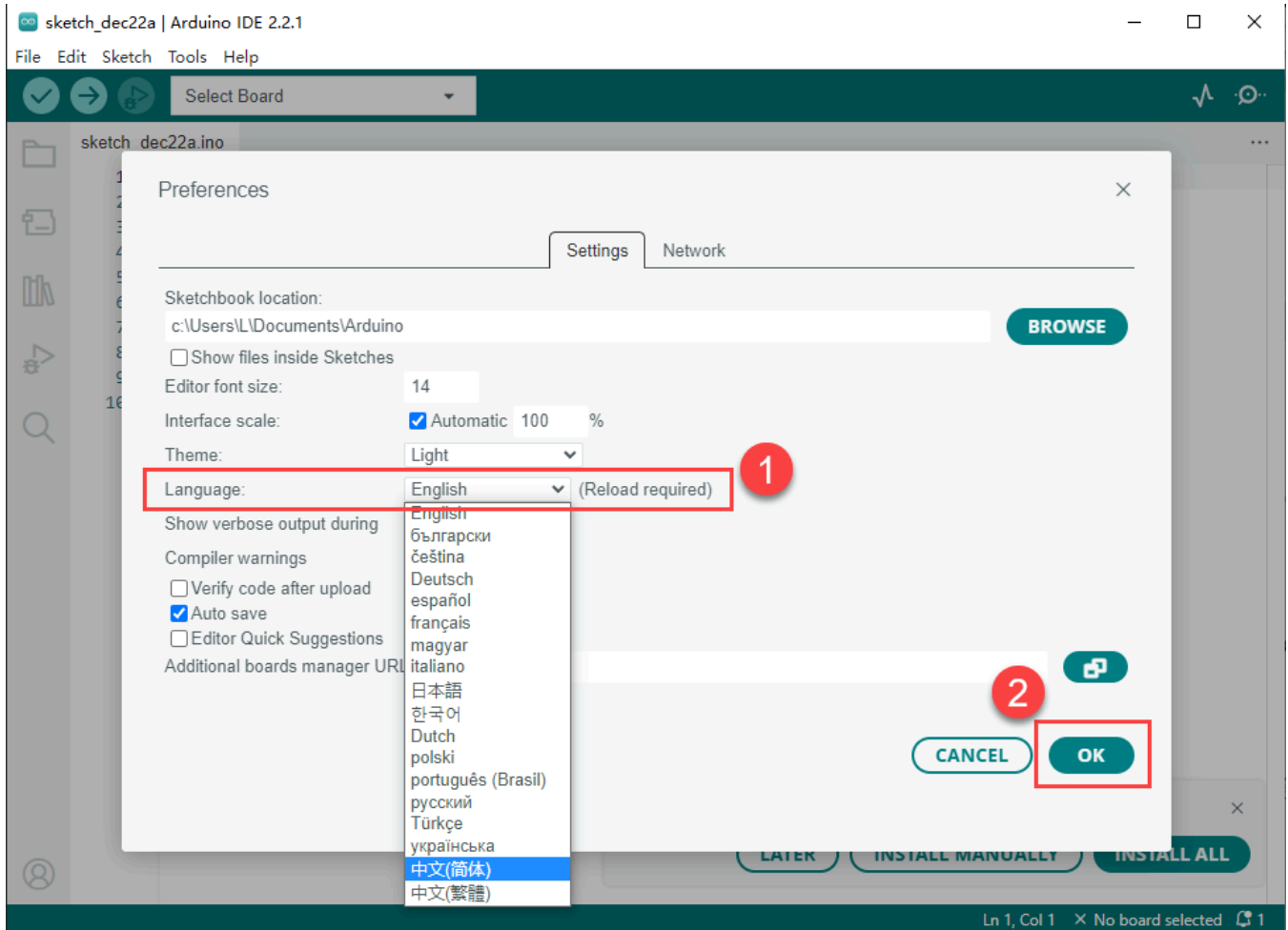
图标	功能
	调试，部分开发板可以通过Arduino IDE实时调试
	选择开发板，可以选择不同的开发板进行项目开发
	串口绘图仪，可以将打印到Arduino串口的数据绘制成图表
	串口监视器，打印串口信息

- 3. **编辑区**：编辑代码的区域。
- 4. **状态栏**：显示当前编辑器的一些状态，例如代码的行列、开发板的信息等。
- 5. **侧边栏**：Arduino IDE的核心，负责显示工作文件夹、代码调试、库文件安装等。

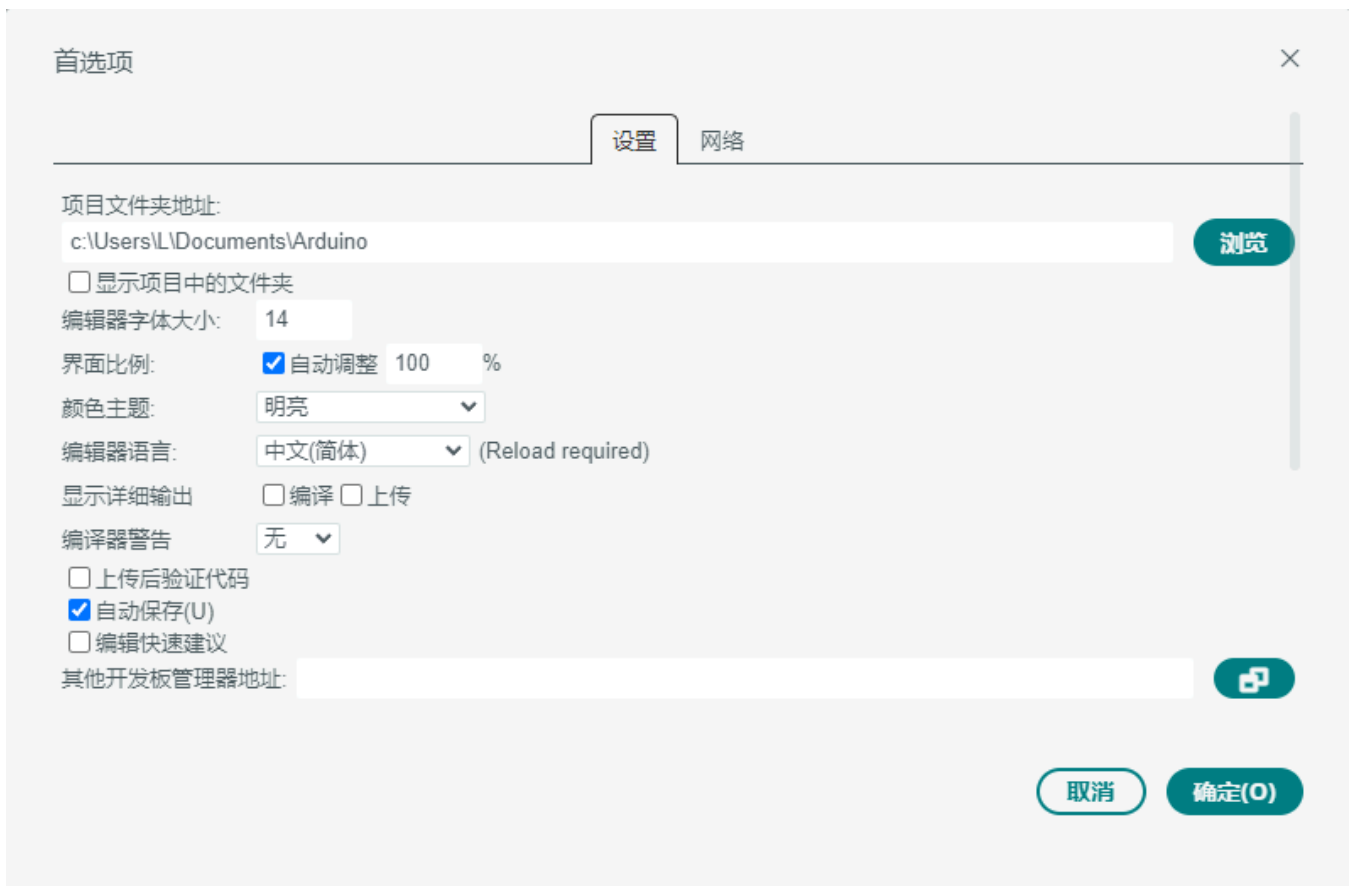
图标	功能
	项目文件夹，显示当前项目的文件。
	开发板管理器，添加开发板工具包。
	库管理，添加或删除程序的库文件。
	调试，对项目实时调试。
	搜索，可搜索或替换代码或变量。

2.1.3 Arduino IDE界面设置

- 1. 修改中文界面：在 Arduino IDE 界面选择“File”->“Preferences”，在弹出的窗口中“language”选项栏中，选择切换为中文，随后点击“OK”即可。

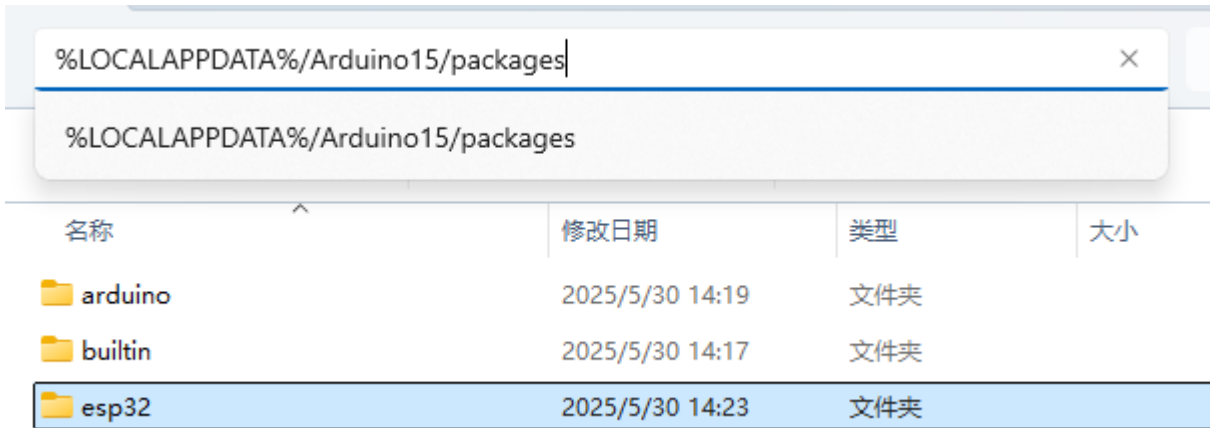


2. 我们可以通过选择"文件->首选项"在弹出的窗口下修改项目文件路径、编辑器文字大小、颜色主题等设置。



2.1.4 ESP32芯片包安装

- 1.如果安装过除**2.0.11**或**2.0.12**版本的其他版本的esp32 package，请先删除。若未安装过，可直接跳转到步骤3。
- 2.删除方法：文件管理器地址栏输入 **%LOCALAPPDATA%/Arduino15/packages**，回车进入，然后删除掉其中的esp32文件夹。



- 3.双击文档同路径下**“ArduinoIDE安装包及芯片包”**文件夹里的**“esp32_package_2.0.11_arduino.exe”**文件，等待安装完成即可。



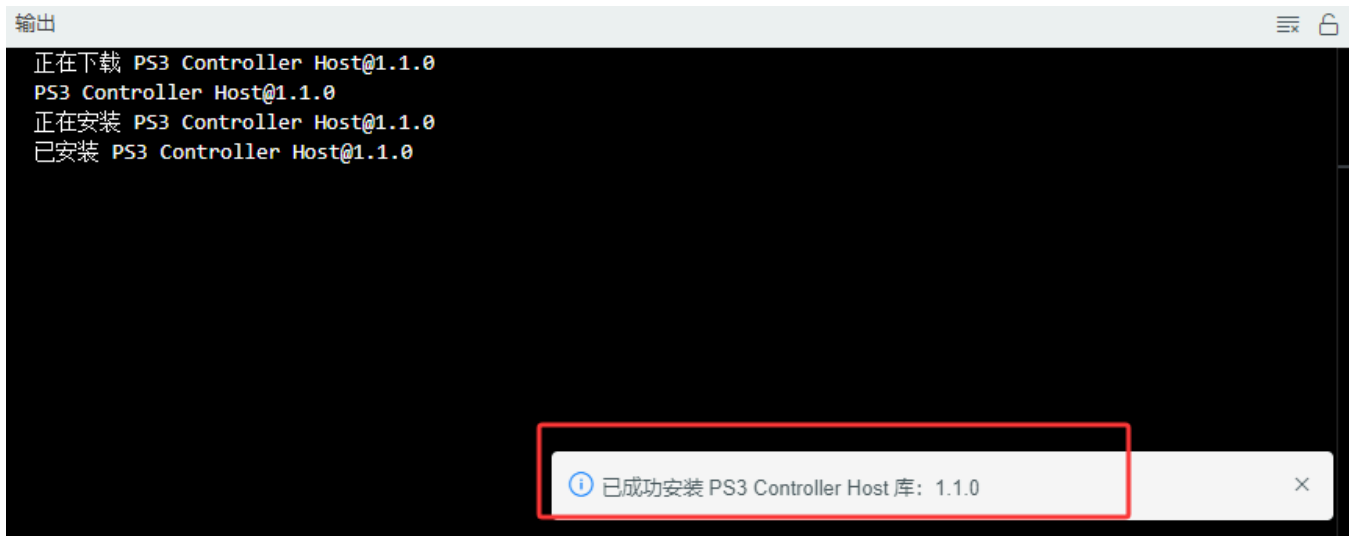
2.1.5 库文件导入（必看！！）

后续程序中需要使用到**PS3 Controller Host**、**MultiButton**和**U8g2**的官方库。在库管理界面分别输入**PS3 Controller Host**、**MultiButton**和**U8g2**，点击安装即可，如下图所示：

- 1. 在Arduino IDE界面左侧的侧边栏处点击 。
- 2. 在弹出的库管理栏中输入**“PS3 Controller Host”**就会自动搜索库文件，然后点击**“安装”**即可。



- 3. 弹出如下提示，则表示安装成功。**MultiButton**和**U8g2**库的安装方法同上。



2.1.6 Arduino程序下载

1. 这里我们以一个打印"hiwonder"字样的例程为例进行说明。在本文档同目录下**02 程序源码\01 Arduino安装包及开发包\Demo**文件夹中双击打开"**Demo.ino**"的示例程序。



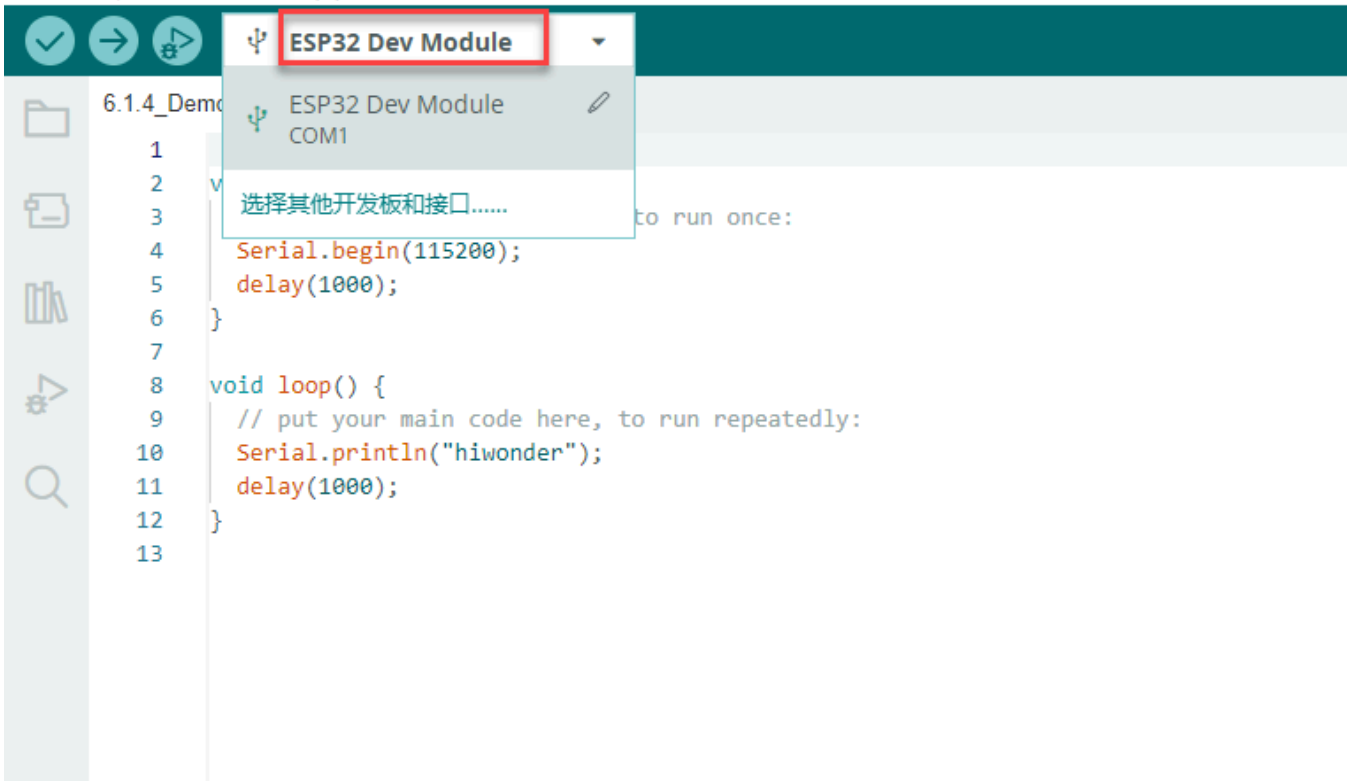
2. 将NexArm通过数据线连接至电脑。



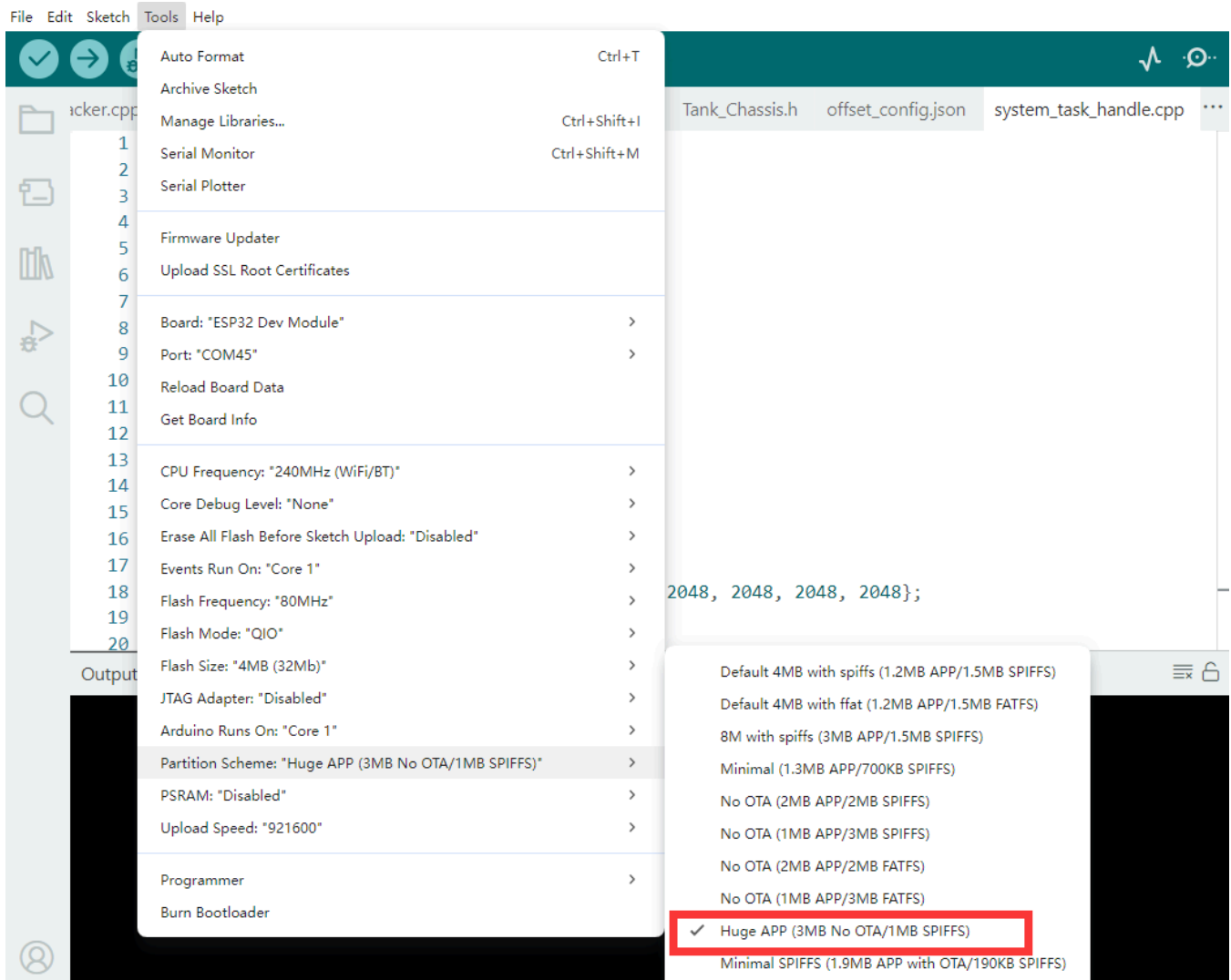
3. 在"选择开发板"选项中找到Arduino对应的开发板。(COM口不唯一，我们可以通过电脑的设备管理中查看COM号，此处以COM6为例)。

6.1.4_Demo | Arduino IDE 2.2.1

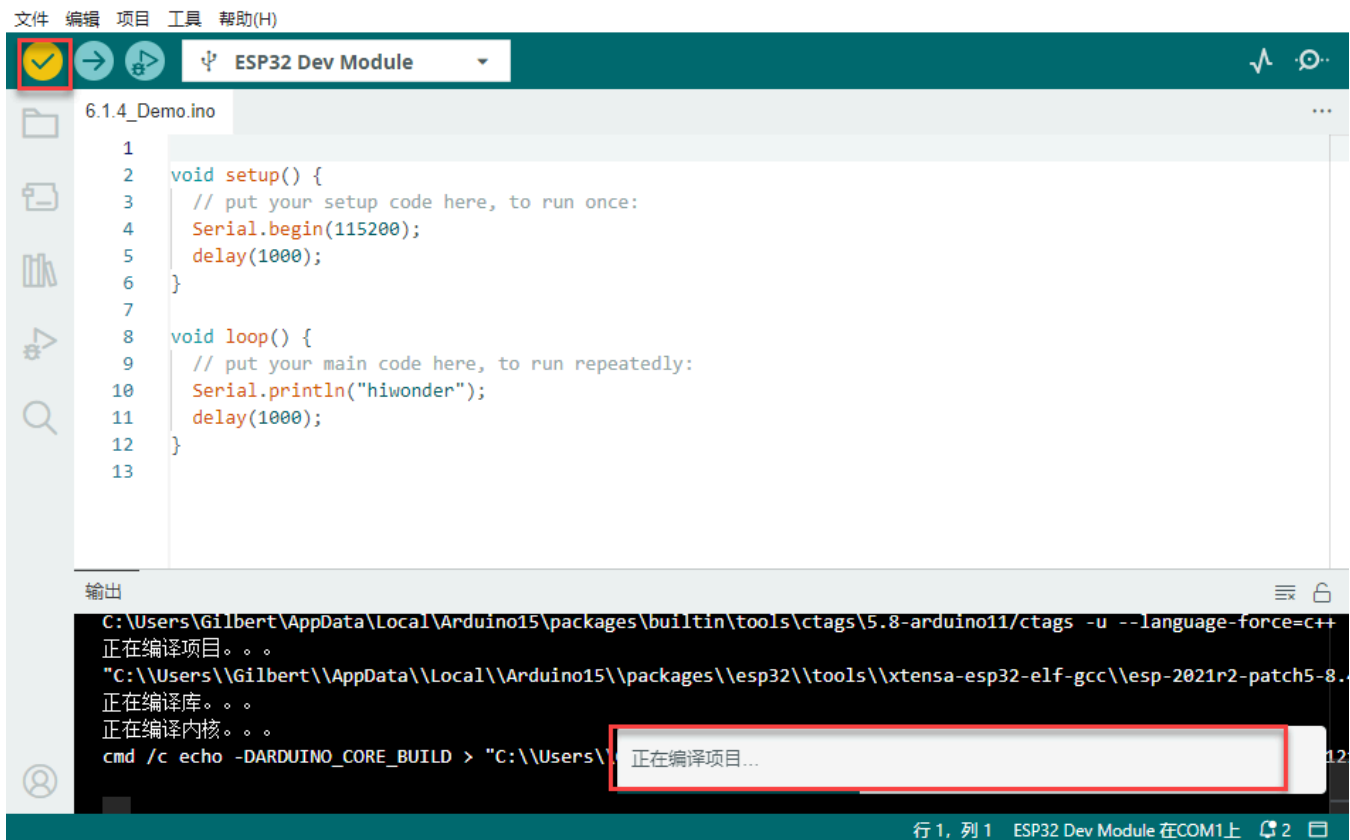
文件 编辑 项目 工具 帮助(H)



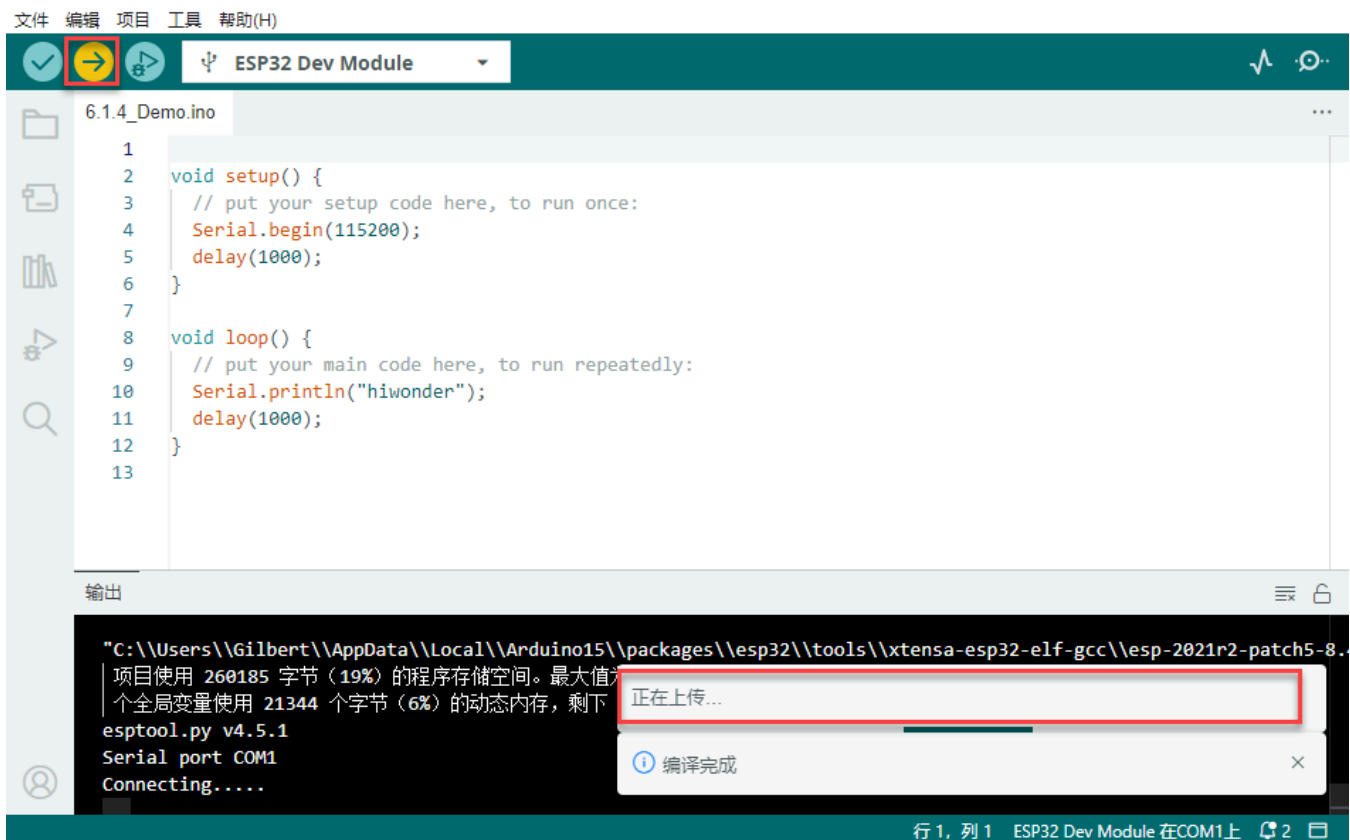
4. 点击菜单栏的"工具", 按照下图选择相应的ESP32的开发板配置:



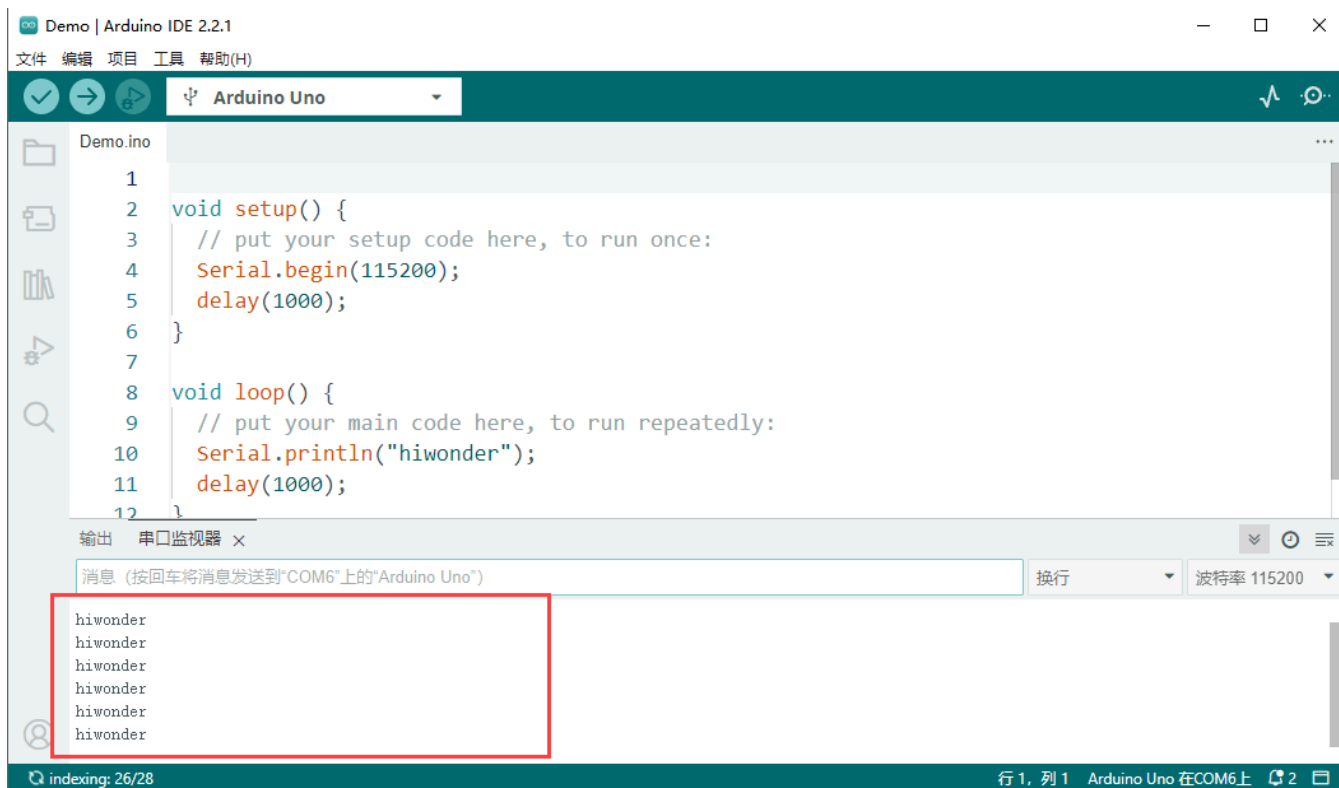
5. 点击  按钮可对程序进行编译，可以验证程序是否存在语法错误等问题。



6. 编译成功后，再点击  按钮会将程序上传到ESP32核心板上。



7. 上传完成后，点击  打开串口监视器，我们可以看到在串口监视器中打印出了"hiwonder"字样。

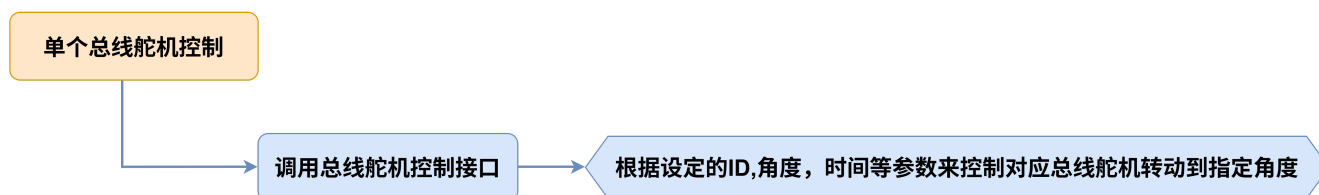


2.2 单个总线舵机控制

2.2.1 项目介绍

本节课程主要演示通过我们封装好的总线舵机控制函数来控制单个总线舵机转动，帮助用户了解总线舵机控制流程和相关接口调用。

2.2.2 实验流程



2.2.3 模块说明

NexArm主要使用了HX-30HM和HX-65HM两款磁编码舵机，下面是这两款舵机的简介。



- HX-30HM**是幻尔推出的一款磁编码总线舵机，集伺服驱动、电机与总线信号于一体，采用半双工UART异步串行接口通信，通信波特率高达1000000bps，通过串口指令实现精确控制。该舵机工作电压范围为9~12.6V，额定11.1V，整机重量仅52g，尺寸紧凑（45.2×24.7×35mm），在11.1V电压下可输出30KG·cm的额定扭矩，转速为0.19sec/60°，支持完整的360°全角度转动，控制精度达0.3°，位置值范围0~4095对应0~360°。在功能特性方面，该舵机支持位置模式、闭环速度模式与开环速度模式三种工作模式灵活切换，内置磁编码器实现角度实时回读，每个舵机配备三个接口可总线串联最多253个舵机协同工作，配置掉电保存、堵转保护及过温保护等多重安全机制，并支持实时反馈温度、电压、位置等关键运行参数。



- HX-65HM**是幻尔推出的一款双轴磁编码总线舵机，集伺服驱动、电机与总线信号于一体，采用半双工UART异步串行接口通信，波特率为1000000，通过串口指令实现精确控制。该舵机工作电压范围为9~12.6V，额定11.1V，重量143.5g，尺寸紧凑（45×25×70mm），在12.6V电压下可输出高达65KG·cm的峰值扭矩，转速为0.19sec/60°，具备完整的360°全角度转动能力，控制精度可达0.3°，位置值范围0~4095对应0~360°。在功能特性方面，该舵机支持位置模式、闭环速度模式与开环速度模式三种工作模式灵活切换，内置磁编码器实现角度实

时回读，可通过总线串联最多253个舵机同步控制，配置掉电保存、过温保护及掉电保护等安全机制，并支持实时反馈温度、电压、位置、电流、速度等多项运行参数。

2.2.4 程序下载

找到本文档同目录下的“02 程序源码”文件夹中的“02 单个总线舵机控制\busservo_control”，打开“busservo_control.ino”，具体下载步骤可参考[Arduino程序下载](#)。

2.2.5 运行效果

程序上传成功后，对应ID的舵机会先转动到2048中位的角度，延时2秒后，在1500和3000的角度位置循环转动，串口会打印相关提示信息。



```
Move servo 5 to pos 3000
Move servo 5 to pos 900
Move servo 5 to pos 3000
Move servo 5 to pos 900
[heartbeat] servo=5 forward=0 last_move_ms=11230
Move servo 5 to pos 3000
Move servo 5 to pos 900
Move servo 5 to pos 3000
Move servo 5 to pos 900
[heartbeat] servo=5 forward=0 last_move_ms=17230
Move servo 5 to pos 3000
```

2.2.6 程序分析

1. `setup()` 中先创建事件循环，再调用 `register_system_task()` 完成初始化；`loop()` 中持续调用 `system_loop_handler()`，因此单个舵机的位置切换就是在这个主循环中不断推进的。

```
#include "Arduino.h"
#include "system_task_handle.h"

esp_event_loop_handle_t loop_handle;

void setup()
{
    esp_event_loop_args_t loop_args = {
        .queue_size = 10,
        .task_name = "sys_evt_loop",
        .task_priority = uxTaskPriorityGet(NULL),
        .task_stack_size = 4096,
        .task_core_id = tskNO_AFFINITY
    };

    esp_event_loop_create(&loop_args, &loop_handle);
    register_system_task(&loop_handle);
}
```

```

}

void loop()
{
    system_loop_handler();
}

```

2. 主程序开头先定义关键参数，包括舵机 ID、通信波特率、加速度、速度以及三个目标位置。后面无论是回中位还是左右摆动，都是围绕这些常量展开的。

```

namespace {
constexpr uint8_t  SERVO_ID      = 5;
constexpr uint32_t SERVO_BAUD    = 1000000;
constexpr uint8_t  ACC           = 80;
constexpr uint16_t SPEED         = 1200;
constexpr uint16_t POS_LOW       = 900;
constexpr uint16_t POS_HIGH      = 3000;
constexpr uint16_t POS_RESET     = 2048;
constexpr uint32_t MOVE_SETTLE_MS = 1500;
constexpr uint32_t RESET_DELAY_MS = 2000;

SerialServo_t servo;
bool target_high = true;
unsigned long last_action_ms = 0;
}

```

3. 初始化中先打开调试串口，再初始化总线舵机串口，然后通过 `set_torque(true)` 给舵机上力矩，最后把舵机转到中位 2048。这样后续实验开始时，舵机总是从统一的初始位置出发。

```

void register_system_task(esp_event_loop_handle_t* loop_handle)
{
    (void)loop_handle;
    Serial.begin(115200);
    delay(50);
    Serial.println("[SingleServo] start");

    servo.begin(Serial1, SERVO_BAUD, SERVO_TX_PIN, SERVO_RX_PIN);
    set_torque(true);

    servo.write_pos_ex(SERVO_ID, ACC, SPEED, POS_RESET);
    last_action_ms = millis();
}

```

4. `set_torque()` 负责控制舵机力矩开关。这里通过广播 ID `0xFE` 向总线发送一次写寄存器命令，把力矩开关寄存器写成 0 或 1，这样就不需要单独为每个舵机再写一套接口。

```
static void set_torque(bool enable)
{
    uint8_t data[2];
    data[0] = ADDR_TORQUE_SWITCH;
    data[1] = enable ? 1 : 0;
    servo.tx_frame_write(BROADCAST_ID, CMD_WRITE, data, 2);
    delay(20);
}
```

5. `system_loop_handler()` 是动作循环的核心。程序先清理串口缓冲，再根据时间间隔判断是否需要发送新的位置命令；当等待时间到达后，舵机会在 `POS_HIGH` 和 `POS_LOW` 两个目标位置之间来回切换。

```
void system_loop_handler()
{
    while (servo.uart && servo.uart->available()) {
        servo.uart->read();
    }

    unsigned long now = millis();
    if (now - last_action_ms < (target_high ? RESET_DELAY_MS : MOVE_SETTLE_MS)) {
        delay(10);
        return;
    }

    uint16_t target = target_high ? POS_HIGH : POS_LOW;
    servo.write_pos_ex(SERVO_ID, ACC, SPEED, target);
    target_high = !target_high;
    last_action_ms = now;
}
```

6. `SerialServo_t::write_pos_ex()` 封装了单个舵机位置控制接口。函数把加速度、目标位置和速度依次写入发送缓存，最后调用 `tx_frame_write()` 发送完整数据帧。应用层只需要传入舵机 ID 和目标参数，不需要自己拼底层协议。

```
void SerialServo_t::write_pos_ex(uint8_t id, uint8_t acc, uint16_t speed, uint16_t pos)
{
    uint8_t data[8];
    data[0] = 0x29;
    data[1] = acc;
    data[2] = pos;
    data[3] = pos >> 8;
    data[4] = 0;
    data[5] = 0;
    data[6] = speed;
    data[7] = speed >> 8;
    tx_frame_write(id, 3, data, 8);
}
```

7. `tx_frame_write()` 是总线协议发送的底层接口。函数按“帧头 + ID + 长度 + 指令 + 参数 + 校验”的格式组织完整数据包，再通过串口发送出去。单舵机控制和多舵机同步控制最后都会落到这一层，因此这个函数就是理解总线协议格式的关键。

```
void SerialServo_t::tx_frame_write(uint8_t id, uint8_t cmd, uint8_t* data, uint8_t len)
{
    uint8_t check = id + len + 2 + cmd;

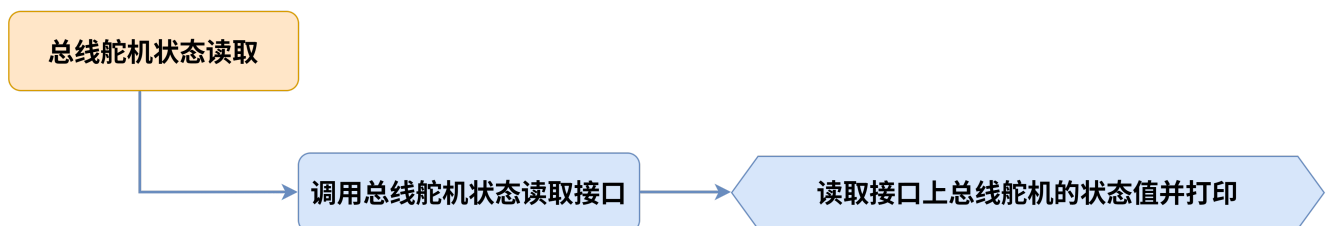
    uart->write(0xFF);
    uart->write(0xFF);
    uart->write(id);
    uart->write(len + 2);
    uart->write(cmd);
    for (int i = 0; i < len; i++) {
        uart->write(data[i]);
        check += data[i];
    }
    uart->write(~check);
}
```

2.3 总线舵机状态读取

2.3.1 项目介绍

本节课程主要演示通过我们封装好的总线舵机状态读取函数来读取总线舵机状态信息并打印，帮助用户了解总线舵机状态读取流程和相关接口调用。

2.3.2 实验流程



2.3.3 模块说明

可跳转至[2.2.3 模块说明](#)查看。

2.3.4 程序下载

找到本文档同目录下的“02 程序源码”文件夹中的“总线舵机位置读取\bus servo_read”，打开“bus servo_read.ino”，具体下载步骤可参考[Arduino程序下载](#)。

2.3.5 运行效果

程序启动后会先对总线上的舵机进行扫描。如果扫描到在线舵机，串口会依次输出每个 ID 舵机的当前角度和坐标信息。



```
输出 串口监视器 x
消息 (按回车将消息发送到“COM3”上的“ESP32 Dev Module”)

ID 1: 2047
ID 2: 2148
ID 3: 927
ID 4: 1521
ID 5: 2048
ID 6: 2048
X Y Z (mm): 105 0 106
Pitch(): -43.4
Roll / Claw (): 0 0
```

2.3.6 程序分析

1. `setup()` 中先创建事件循环，再调用 `register_system_task()` 完成串口、舵机总线和状态读取模块初始化；`loop()` 中持续调用 `system_loop_handler()`，因此 PING 扫描、寄存器读取和 AT32 回退打印都是在这个主循环中不断推进的。

```
#include "Arduino.h"
#include "system_task_handle.h"

esp_event_loop_handle_t loop_handle;

void setup()
{
    esp_event_loop_args_t loop_args = {
        .queue_size = 10,
        .task_name = "sys_evt_loop",
        .task_priority = uxTaskPriorityGet(NULL),
        .task_stack_size = 4096,
        .task_core_id = tskNO_AFFINITY
    };

    esp_event_loop_create(&loop_args, &loop_handle);
    register_system_task(&loop_handle);
}

void loop()
{
    system_loop_handler();
}
```

2. 主程序开头先定义了舵机通信波特率、读取周期、AT32 轮询周期、状态打印周期、固定 ID 列表以及在线舵机缓存。后续程序会先尝试 PING 扫描在线舵机，如果没有发现舵机，再根据这些标志位切换到 AT32 回读模式。

```

static const uint32_t SERVO_BAUD = 1000000;
static const uint32_t READ_INTERVAL_MS = 2000;
static const uint32_t AT32_POLL_MS = 150;
static const uint32_t STATUS_PRINT_MS = 2000;

static const bool USE_PING_DISCOVER = true;
static const uint8_t FIXED_SERVO_IDS[] = { 4, 5, 6 };
static const bool AUTO_AT32_FALLBACK = true;

#define MAX_TRACKED_SERVOS 16
static uint8_t online_ids[MAX_TRACKED_SERVOS];
static uint8_t online_count = 0;
static bool g_at32_poll_mode = false;

```

3. `register_system_task()` 先完成串口、舵机总线和力矩初始化，然后根据 `USE_PING_DISCOVER` 决定是扫描总线还是直接装载固定 ID 列表。若没有任何舵机通过 PING，程序会自动启用 `CommProtocol`，转而请求板载 AT32 返回当前位姿和关节脉冲值。

```

void register_system_task(esp_event_loop_handle_t *event_loop)
{
    (void)event_loop;

    Serial1.begin(115200);
    delay(200);

    servo.begin(Serial1, SERVO_BAUD, SERVO_TX_PIN, SERVO_RX_PIN);
    delay(50);

    flush_servo_rx();
    set_torque(true);
    delay(AFTER_TORQUE_MS);
    flush_servo_rx();

    uint16_t at32_like = 0;
    if (USE_PING_DISCOVER)
        run_ping_discovery(&at32_like);
    else
        load_fixed_id_list();

    g_at32_poll_mode = (online_count == 0 && AUTO_AT32_FALLBACK);
    if (g_at32_poll_mode) {
        g_bus_proto.begin();
        g_bus_proto.register_success_callback(at32_packet_callback);
        request_at32_poll_one();
    }

    last_read_ms = millis();
}

```

4. `run_ping_discovery()` 用于扫描指定 ID 范围内的舵机。函数会依次向各个 ID 发送 `CMD_PING`，若舵机正确回包，就把这个 ID 存入 `online_ids[]` 中。这样后续读取阶段就只需要轮询已经确认在线的舵机。

```

static void run_ping_discovery(uint16_t *out_at32_like_count)
{
    online_count = 0;
    memset(online_ids, 0, sizeof(online_ids));
    *out_at32_like_count = 0;

    for (uint8_t id = PING_SCAN_ID_MIN; id <= PING_SCAN_ID_MAX; id++) {
        flush_servo_rx();
        delay(PING_GAP_MS);

        bool ok = servo.ping(id);
        if (ok) {
            if (online_count < MAX_TRACKED_SERVOS)
                online_ids[online_count++] = id;
        } else {
            if (servo.last_read_err == -4 && servo.last_read_rx_id == AT32_RESP_ID &&
                servo.last_read_bytes >= 6)
                (*out_at32_like_count)++;
        }
        delay(PING_GAP_MS);
    }
}

```

5. 当程序进入 AT32 回退模式后，`at32_packet_callback()` 会专门解析 `CMD_GET_CUR_COORDS` 和 `CMD_GET_REAL_JOINT_ANGLES` 两类回包。前者提取末端 `x、y、z、pitch、roll、claw`，后者提取 6 路关节脉冲值，这样即使总线舵机不直接回包，也能通过板载 AT32 获取整机状态。

```

static void at32_packet_callback(PacketTypeDef *rx_packet)
{
    if (rx_packet->elements.id != AT32_RESP_ID)
        return;

    if (rx_packet->elements.cmd == CMD_GET_CUR_COORDS &&
        rx_packet->elements.length >= 26) {
        g_snap_x = (int16_t)((rx_packet->elements.args[1] << 8) | rx_packet->
>elements.args[0]);
        g_snap_y = (int16_t)((rx_packet->elements.args[3] << 8) | rx_packet->
>elements.args[2]);
        g_snap_z = (int16_t)((rx_packet->elements.args[5] << 8) | rx_packet->
>elements.args[4]);
        g_snap_pitch_x10 = (int16_t)((rx_packet->elements.args[7] << 8) | rx_packet->
>elements.args[6]);
        g_snap_roll = (int16_t)((rx_packet->elements.args[9] << 8) | rx_packet->
>elements.args[8]);
        g_snap_claw = (int16_t)((rx_packet->elements.args[11] << 8) | rx_packet->
>elements.args[10]);
        return;
    }

    if (rx_packet->elements.cmd == CMD_GET_REAL_JOINT_ANGLES &&
        rx_packet->elements.length >= 26) {
        for (int i = 0; i < 6; i++) {

```

```

        int16_t pulse = (int16_t)((rx_packet->elements.args[i * 4 + 1] << 8) |
                                   rx_packet->elements.args[i * 4]);
        g_snap_pulse[i] = pulse;
    }
}
}

```

6. `SerialServo_t::ping()`、`read_reg()` 和 `read_servo_status()` 共同构成了总线舵机状态读取链路。
`ping()` 用于确认某个 ID 是否在线，`read_reg()` 负责按寄存器地址读取指定字节数，
`read_servo_status()` 则在此基础上进一步解析位置、速度、负载、电压、温度、电流、中位偏差和工作模式。

```

bool SerialServo_t::ping(uint8_t id)
{
    uint8_t req[] = { FRAME_HEADER_1, FRAME_HEADER_2, id, 2, CMD_PING };
    uint8_t ck = data_check(req, 5);
    uart->write(req, 5);
    uart->write(&ck, 1);
    ...
}

int SerialServo_t::read_reg(uint8_t id, uint8_t start_addr, uint8_t count, uint8_t *out)
{
    uint8_t req[2] = { start_addr, count };
    tx_frame_write(id, CMD_READ, req, 2);
    ...
}

bool SerialServo_t::read_servo_status(uint8_t id, ServoStatus_t *status)
{
    uint8_t raw[16];
    int n = read_reg(id, REG_PRESENT_POSITION_L, 15, raw);
    if (n != 15)
        return false;
    ...
}

```

7. `print_servo_by_hx_protocol()` 把舵机状态读取结果统一打印成可读信息。函数先调用
`read_servo_status()` 读取主状态块，再额外读取目标位置、目标速度等寄存器，最后把角度、速度、负载、
电压、温度和工作模式整理后输出到串口。

```

static void print_servo_by_hx_protocol(uint8_t id)
{
    ServoStatus_t st;
    if (!servo.read_servo_status(id, &st)) {
        Serial.printf("[ID=%u] CMD_READ 失败 err=%d rx=%u\n", (unsigned)id,
                      servo.last_read_err, (unsigned)servo.last_read_bytes);
        return;
    }

    float deg = (POS_TICKS_PER_TURN > 0.1f)

```

```

        ? ((float)st.position * 360.0f / POS_TICKS_PER_TURN)
        : 0.f;

Serial.printf("===== HX-30HM ID=%u =====\n", (unsigned)id);
Serial.printf(" [Reg56-57] 当前位置: %d (≈%.2f°)\n", (int)st.position, deg);
Serial.printf(" [Reg58-59] 速度: %d [Reg60-61]负载: %d\n", (int)st.speed,
(int)st.load);
Serial.printf(" [Reg62] 电压: %.1f V [Reg63] 温度: %u °C\n",
st.voltage * 0.1f, (unsigned)st.temperature);
}

```

8. `system_loop_handler()` 中根据当前模式走两条不同路径: 如果启用了 `g_at32_poll_mode`, 程序就持续把串口字节送入协议解析器, 并按固定节拍请求位姿和关节角; 如果已经扫描到在线舵机, 则按 `READ_INTERVAL_MS` 周期依次读取每个在线舵机的状态并打印出来。

```

void system_loop_handler(void)
{
    if (g_at32_poll_mode) {
        if (servo.uart) {
            while (servo.uart->available()) {
                uint8_t c = servo.uart->read();
                g_bus_proto.parsing(&c, 1);
            }
        }
        uint32_t now = millis();
        if ((uint32_t)(now - g_last_at32_poll_ms) >= AT32_POLL_MS) {
            g_last_at32_poll_ms = now;
            request_at32_poll_one();
        }
        if ((uint32_t)(now - g_last_status_print_ms) >= STATUS_PRINT_MS) {
            g_last_status_print_ms = now;
            emit_at32_status_block();
        }
        delay(1);
        return;
    }

    if (millis() - last_read_ms < READ_INTERVAL_MS) {
        delay(1);
        return;
    }
    last_read_ms = millis();

    for (uint8_t i = 0; i < online_count; i++)
        print_servo_by_hx_protocol(online_ids[i]);
}

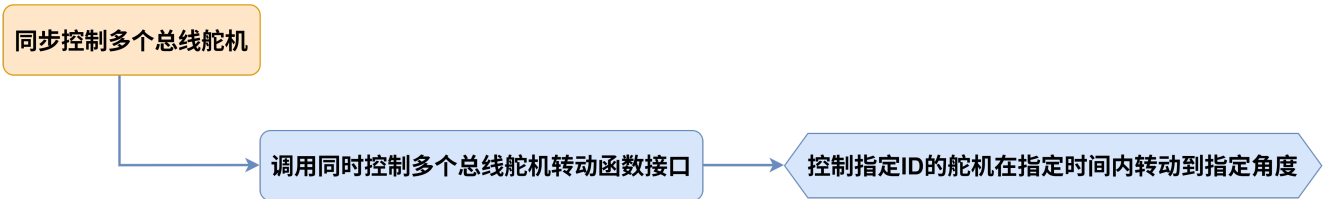
```

2.4 同步控制多个总线舵机

2.4.1 项目介绍

本节课程主要演示通过我们封装好的多个总线舵机同步控制函数来控制多个总线舵机同时转到指定角度，帮助用户了解多个总线舵机同步控制流程和相关接口调用。

2.4.2 实验流程



2.4.3 模块说明

可跳转至[2.2.3 模块说明](#)查看。

2.4.4 程序下载

找到本文档同目录下的“02 程序源码”文件夹中的“04 同步控制多个总线舵机\multi_servo_control”，打开“multi_servo_control.ino”，具体下载步骤可参考[Arduino程序下载](#)。

2.4.5 运行效果

程序执行后机械臂先回到整机约定的默认伸展位，稍作停顿后再进入循环：多台舵机在同一时刻按设定速度一起动作，依次在三种姿态之间反复切换，每一步之间留有足够时间等舵机到位。打开串口监视器，会看到启动提示、默认位就绪与开始循环等说明。

输出 串口监视器 x

消息 (按回车将消息发送到“COM3”上的“ESP32 Dev Module”)

```
Starting pose cycle (A -> B -> C only).
Move all servos to POSE_A
Move all servos to POSE_B
Move all servos to POSE_C
Move all servos to POSE_A
Move all servos to POSE_B
Move all servos to POSE_C
```

2.4.6 程序分析

1. `setup()` 中创建事件循环并调用 `register_system_task()` 完成初始化；`loop()` 中持续执行 `system_loop_handler()`，因此多个舵机的同步切换动作也是依靠主循环不断调度完成的。

```
#include "Arduino.h"
```

```
#include "system_task_handle.h"

esp_event_loop_handle_t loop_handle;

void setup()
{
    esp_event_loop_args_t loop_args = {
        .queue_size = 10,
        .task_name = "sys_evt_loop",
        .task_priority = uxTaskPriorityGet(NULL),
        .task_stack_size = 4096,
        .task_core_id = tskNO_AFFINITY
    };

    esp_event_loop_create(&loop_args, &loop_handle);
    register_system_task(&loop_handle);
}

void loop()
{
    system_loop_handler();
}
```

2. 主程序开头定义了同步控制实验的关键参数，包括两个舵机 ID、同步速度、动作等待时间以及三组目标姿态。这里通过数组保存姿态数据，后面切换动作时只需要改变数组索引即可。

```
namespace {
constexpr uint8_t  SERVO_IDS[] = {5, 6};
constexpr uint8_t  SERVO_NUM   = sizeof(SERVO_IDS) / sizeof(SERVO_IDS[0]);
constexpr uint32_t SERVO_BAUD  = 1000000;
constexpr uint16_t SPEED       = 1200;
constexpr uint32_t MOVE_SETTLE_MS = 2200;
constexpr uint32_t RESET_DELAY_MS = 2000;
constexpr uint32_t ARM_RESET_DURATION_MS = 2000;

const int16_t POSE_A[SERVO_NUM] = {2048, 2048};
const int16_t POSE_B[SERVO_NUM] = {760, 1000};
const int16_t POSE_C[SERVO_NUM] = {3300, 2800};
const int16_t* POSES[] = {POSE_A, POSE_B, POSE_C};

SerialServo_t servo;
Robot_Arm arm;
uint8_t current_pose_idx = 0;
bool reset_done = false;
unsigned long last_reset_done_ms = 0;
}
```

3. 文件中先定义了三个辅助函数。`move_arm_to_default_pose()` 用于让整机回到默认位，`set_torque()` 负责统一开启舵机力矩，`move_to_pose()` 则负责把一组目标位置通过同步写命令一次性发给多个舵机。

```
static void move_arm_to_default_pose()
{
```

```

    arm.reset_all(ARM_RESET_DURATION_MS);
}

static void set_torque(bool enable)
{
    uint8_t data[2];
    data[0] = ADDR_TORQUE_SWITCH;
    data[1] = enable ? 1 : 0;
    servo.tx_frame_write(BROADCAST_ID, CMD_WRITE, data, 2);
    delay(20);
}

static void move_to_pose(const int16_t* pose)
{
    servo.sync_write_pos_speed(SERVO_IDS, pose, SPEED, SERVO_NUM);
}

```

4. 初始化中先完成调试串口、总线舵机串口和机械臂对象初始化，然后开启舵机力矩，最后调用 `move_arm_to_default_pose()` 让机械臂回到默认位。这样后面的同步控制就都从统一姿态开始。

```

void register_system_task(esp_event_loop_handle_t* loop_handle)
{
    (void)loop_handle;
    Serial1.begin(115200);
    delay(50);
    Serial1.println("[MultiServo] start");

    servo.begin(Serial1, SERVO_BAUD, SERVO_TX_PIN, SERVO_RX_PIN);
    arm.begin();

    set_torque(true);
    move_arm_to_default_pose();

    last_reset_done_ms = millis();
    reset_done = true;
}

```

5. `system_loop_handler()` 是主状态调度函数。程序先等待默认位动作执行完成，再按顺序切换到 `POSE_A`、`POSE_B`、`POSE_C` 三组姿态。每次发送同步写命令后都会等待一段时间，确保多个舵机有足够的运行时间完成动作。

```

void system_loop_handler()
{
    while (servo.uart && servo.uart->available()) {
        servo.uart->read();
    }

    unsigned long now = millis();
    if (!reset_done) {
        delay(10);
        return;
    }
}

```

```

if (now - last_reset_done_ms < RESET_DELAY_MS && current_pose_idx == 0) {
    delay(10);
    return;
}

static unsigned long last_move_ms = 0;
if (now - last_move_ms < MOVE_SETTLE_MS) {
    delay(10);
    return;
}

move_to_pose(POSES[current_pose_idx]);
current_pose_idx = (current_pose_idx + 1) % (sizeof(POSES) / sizeof(POSES[0]));
last_move_ms = now;
}

```

6. `Robot_Arm::reset_all()` 用于让整机先回到默认位。函数内部本质上还是调用 `move()` 发送一组默认末端位姿，因此这个实验在开始做多舵机同步之前，先通过运动学接口把机械臂整体调整到便于观察的姿态。

```

bool Robot_Arm::reset_all(int16_t duration_ms)
{
    return move(200.0f, 0.0f, 200.0f, 0.0f, 0.0f, 0.0f, duration_ms);
}

```

7. `SerialServo_t::sync_write_pos_speed()` 是本节的核心接口。函数把多个舵机的 ID、目标位置和速度打包成一帧广播同步写命令，一次性发送到总线上，从而让多个舵机同时开始动作。

```

void SerialServo_t::sync_write_pos_speed(const uint8_t* id, const int16_t* pos, uint16_t
speed, uint8_t num)
{
    uint8_t data[64];
    uint8_t idx = 0;
    data[idx++] = 0x2A;
    data[idx++] = 0x06;
    for (uint8_t i = 0; i < num; i++) {
        data[idx++] = id[i];
        data[idx++] = pos[i];
        data[idx++] = pos[i] >> 8;
        data[idx++] = 0;
        data[idx++] = 0;
        data[idx++] = speed;
        data[idx++] = speed >> 8;
    }
    tx_frame_write(0xFE, 131, data, idx);
}

```

8. 底层的 `tx_frame_write()` 仍然负责真正的协议打包和串口发送。和单个舵机控制相比，这里最大的区别是 ID 使用广播地址 `0xFE`，指令使用同步写命令，因此总线上的多个舵机会在同一时刻解析并执行这一帧数据。

```

void SerialServo_t::tx_frame_write(uint8_t id, uint8_t cmd, uint8_t* data, uint8_t len)

```

```
{
    uint8_t check = id + len + 2 + cmd;

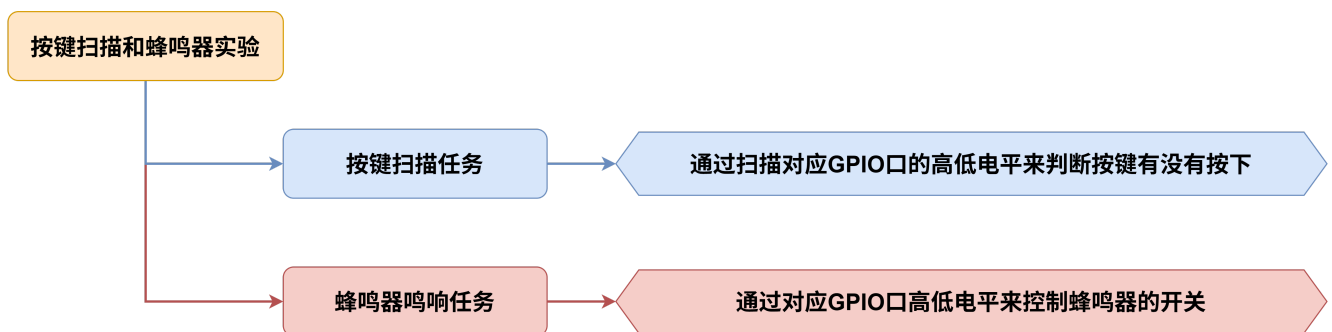
    uart->write(0xFF);
    uart->write(0xFF);
    uart->write(id);
    uart->write(len + 2);
    uart->write(cmd);
    for (int i = 0; i < len; i++) {
        uart->write(data[i]);
        check += data[i];
    }
    uart->write(~check);
}
```

2.5 按键扫描与蜂鸣器

2.5.1 项目介绍

本实验通过 NexArm 控制板上的两个按键触发不同的蜂鸣器提示音，学习按键扫描、状态判断和蜂鸣器驱动的基本方法。程序会在循环中持续更新按键与蜂鸣器状态，当检测到 KEY1 键或 KEY2 键被单击后，分别发出不同频率和次数的提示音，并通过串口输出当前事件。

2.5.2 实验流程



2.5.3 模块说明

- **蜂鸣器**：NexArm搭载的蜂鸣器型号为**BUZZ-8530**，是一款常见的有源电磁式蜂鸣器，工作电压通常为3~5V，兼容3.3V和5V的单片机系统。该蜂鸣器内部集成了振荡驱动电路，只需在正负极施加额定直流电压即可自动发出固定频率的声音，无需外部提供驱动信号，使用极为简便。其工作频率一般在2300Hz左右，空载电流约为30mA以内，声压级可达85dB以上，能够满足大多数提示音、报警音等应用场景的需求。

2.5.6 程序分析

1. `setup()` 中先创建事件循环，再调用 `register_system_task()` 完成初始化；`loop()` 中持续执行 `system_loop_handler()`，因此按键扫描和蜂鸣器状态刷新都依赖这个主循环不断运行。

```
#include "Arduino.h"
#include "system_task_handle.h"

esp_event_loop_handle_t loop_handle;

void setup()
{
    esp_event_loop_args_t loop_args = {
        .queue_size = 10,
        .task_name = "sys_evt_loop",
        .task_priority = uxTaskPriorityGet(NULL),
        .task_stack_size = 4096,
        .task_core_id = tskNO_AFFINITY
    };

    esp_event_loop_create(&loop_args, &loop_handle);
    register_system_task(&loop_handle);
}

void loop()
{
    system_loop_handler();
}
```

2. 主程序开头先在匿名命名空间中定义按键对象、蜂鸣器对象和相关参数。后面无论是触发哪一种提示音，都是围绕这些对象和常量展开的。

```
namespace {
    Button_t g_button;
    Buzzer_t g_buzzer;

    constexpr uint16_t KEY1_BEEP_FREQ = 2000;
    constexpr uint16_t KEY2_BEEP_FREQ = 3000;
    constexpr uint32_t LOOP_DELAY_MS = 10;
}
```

3. `register_system_task()` 负责完成实验启动时的准备工作。程序先初始化串口和蜂鸣器，然后播放一次开机提示音，并通过串口输出当前实验说明。

```
void register_system_task(esp_event_loop_handle_t *event_loop)
{
    (void)event_loop;

    Serial.begin(115200);
    delay(100);
}
```

```

g_buzzer.begin();
g_buzzer.set(60, 0, 1, KEY1_BEEP_FREQ);

Serial.println("simple button + buzzer demo started.");
Serial.println("KEY1 button -> single short beep");
Serial.println("KEY2 button -> double short beep");
}

```

4. `system_loop_handler()` 是实验的核心循环。程序先更新按键状态，再刷新蜂鸣器状态机；如果检测到某个按键被单击，就调用 `set()` 写入蜂鸣器鸣叫时间、间隔时间和次数。

```

void system_loop_handler(void)
{
    g_button.update();
    g_buzzer.update();

    if (g_button.is_clicked(0)) {
        g_buzzer.set(100, 80, 1, KEY1_BEEP_FREQ);
        Serial.println("KEY1 button clicked");
    }

    if (g_button.is_clicked(1)) {
        g_buzzer.set(80, 80, 2, KEY2_BEEP_FREQ);
        Serial.println("KEY2 button clicked");
    }

    delay(LOOP_DELAY_MS);
}

```

5. `Button_t` 对两个按键做了统一封装。`update()` 每次循环都刷新按键状态，`is_clicked()` 则根据传入的编号返回 KEY1 键或 KEY2 键是否产生单击事件。

```

void Button_t::update()
{
    KEY1_Button.update();
    KEY2_Button.update();
}

bool Button_t::is_clicked(uint8_t id)
{
    return id == 0 ? KEY1_Button.isClick() : KEY2_Button.isClick();
}

```

6. `Buzzer_t::begin()` 完成蜂鸣器硬件初始化，`set()` 用来写入本次鸣叫任务的参数。调用 `set()` 后，状态机在后续循环中按设定参数逐步执行这个鸣叫任务。

```

void Buzzer_t::begin()
{
    allow_change = true;

    this->ticks_on = 0;
}

```

```

    this->ticks_off = 0;
    this->times = 0;
    this->freq = 2000;

    stage = BUZZER_STAGE_START_NEW_CYCLE;
    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);
    ledcAttachPin(BUZZER_PIN, 0);
}

bool Buzzer_t::set(uint32_t on_time , uint32_t off_time , uint16_t times, uint16_t freq)
{
    if(allow_change) {
        this->ticks_on = on_time;
        this->ticks_off = off_time;
        this->times = times;
        this->freq = freq;
        this->stage = BUZZER_STAGE_START_NEW_CYCLE;
        return true;
    }
    return false;
}

```

7. `Buzzer_t::update()` 内部使用状态机控制蜂鸣器的鸣叫过程。 `BUZZER_STAGE_START_NEW_CYCLE` 用来启动一次新的响铃， `BUZZER_STAGE_WAITING_OFF` 控制鸣叫持续时间， `BUZZER_STAGE_WAITING_PERIOD_END` 控制两次鸣叫之间的间隔，因此整个提示音过程不需要阻塞式延时。

```

void Buzzer_t::update()
{
    switch(stage) {
        case BUZZER_STAGE_START_NEW_CYCLE:
            if(ticks_on > 0) {
                tone(BUZZER_PIN, freq);
                if(ticks_off > 0) {
                    ticks_count = 0;
                    stage = BUZZER_STAGE_WAITING_OFF;
                } else {
                    stage = BUZZER_STAGE_IDLE;
                }
            } else {
                noTone(BUZZER_PIN);
                stage = BUZZER_STAGE_IDLE;
            }
            break;
        case BUZZER_STAGE_WAITING_OFF:
            ticks_count += update_period;
            if(ticks_count >= ticks_on) {
                noTone(BUZZER_PIN);
                stage = BUZZER_STAGE_WAITING_PERIOD_END;
            }
            break;
        case BUZZER_STAGE_WAITING_PERIOD_END:

```

```

        ticks_count += update_period;
        if(ticks_count >= (ticks_off + ticks_on)) {
            ticks_count -= (ticks_off + ticks_on);
            if(times == 1) {
                noTone(BUZZER_PIN);
                stage = BUZZER_STAGE_IDLE;
            } else {
                tone(BUZZER_PIN, freq);
                times = times == 0 ? 0 : times - 1;
                stage = BUZZER_STAGE_WAITING_OFF;
            }
        }
        break;
    case BUZZER_STAGE_IDLE:
        break;
}
}

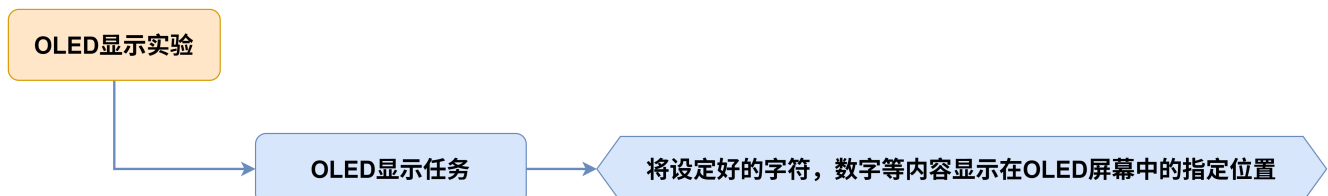
```

2.6 OLED显示

2.6.1 项目介绍

本实验使用板载 OLED 显示屏实时显示文本信息，学习 I2C 设备初始化、文本缓存更新和定时刷新显示的方法。程序启动后会先检测 OLED 是否连接成功，随后周期性更新计数值和系统运行时间，并将这些内容显示到屏幕上。

2.6.2 实验流程



2.6.3 模块说明

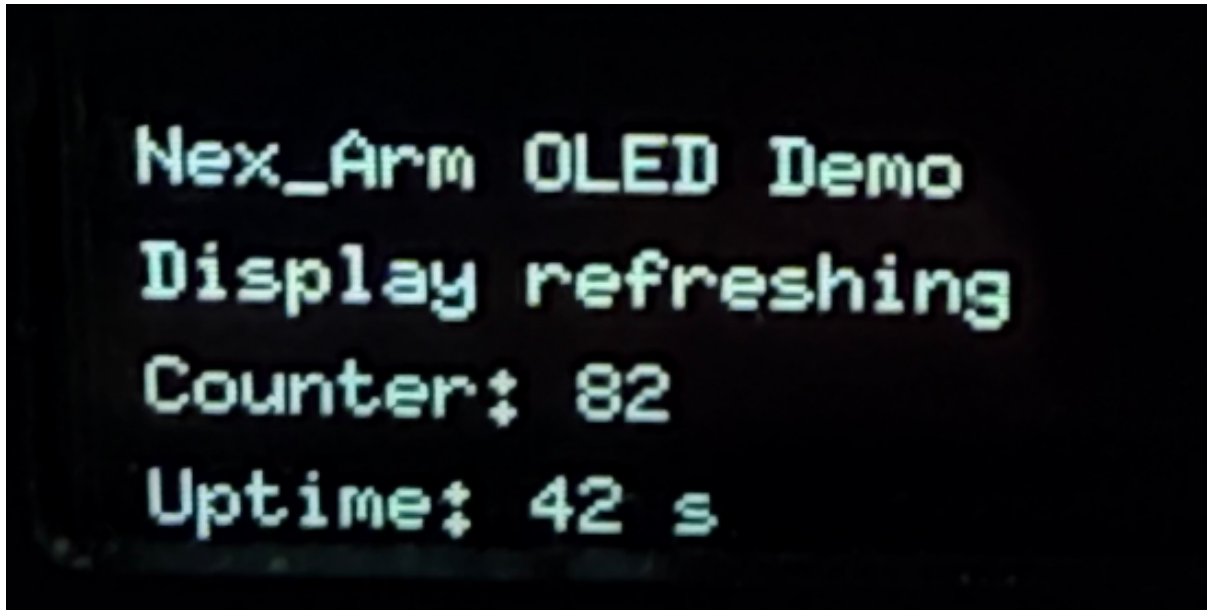
OLED 显示屏采用 SSD1306 控制芯片，分辨率为 128×64 像素，属于单色 OLED 屏。屏幕通过 I2C 总线与主控板通信，OLED 的 I2C 设备地址为 0x3C。硬件连接中，OLED 的 SDA 数据线连接到主控 GPIO26，SCL 时钟线连接到主控 GPIO27，I2C 通信时钟设置为 400kHz。该 OLED 没有单独配置复位引脚，复位引脚参数为未使用。屏幕显示方向设置为 R2，即相对于默认方向旋转 180° 显示。

2.6.4 程序下载

找到本文档同目录下的“02 程序源码”文件夹中的“06 OLED显示\Oled_display”，打开“Oled_display.ino”，具体下载步骤可参考[Arduino程序下载](#)。

2.6.5 运行效果

程序运行后，OLED 屏幕会显示实验标题、当前状态、刷新计数值以及系统运行时间。串口会同步输出刷新次数，方便观察程序是否按设定周期正常运行。



2.6.6 程序分析

1. 主程序开头先定义 OLED 对象、初始化标志位、刷新时间戳和刷新计数器。这些变量共同决定屏幕多久刷新一次，以及当前已经刷新了多少次。

```
namespace {
  OLED_t g_oled;
  bool g_oled_ready = false;
  unsigned long g_last_refresh_ms = 0;
  unsigned long g_counter = 0;

  constexpr unsigned long REFRESH_INTERVAL_MS = 500;
}
```

2. `register_system_task()` 中先调用 `g_oled.begin()` 检查 OLED 是否接入成功。初始化成功后，程序把 4 行要显示的内容先写入缓存，再立即调用 `show_custom()` 完成首次显示。

```
void register_system_task(esp_event_loop_handle_t *event_loop)
{
  (void)event_loop;

  Serial.begin(115200);
  delay(100);

  g_oled_ready = g_oled.begin();
  if (g_oled_ready) {
    Serial.println("OLED demo started.");
    g_oled.set_custom_text(0, "Nex_Arm OLED Demo");
    g_oled.set_custom_text(1, "Display ready");
    g_oled.set_custom_text(2, "Counter: 0");
    g_oled.set_custom_text(3, "Uptime: 0 s");
    g_oled.show_custom();
  } else {
    Serial.println("OLED init failed.");
  }
}
```

```
}
```

3. `system_loop_handler()` 中先判断 OLED 是否初始化成功, 如果未成功就直接返回; 否则每隔 `REFRESH_INTERVAL_MS` 毫秒更新一次计数值和运行时间, 再把最新内容重新写到屏幕上。

```
void system_loop_handler(void)
{
    if (!g_oled_ready) {
        delay(100);
        return;
    }

    unsigned long now = millis();

    if (now - g_last_refresh_ms >= REFRESH_INTERVAL_MS) {
        g_last_refresh_ms = now;
        g_counter++;

        g_oled.set_custom_text(0, "Nex_Arm OLED Demo");
        g_oled.set_custom_text(1, "Display refreshing");
        g_oled.set_custom_text(2, "Counter: " + String(g_counter));
        g_oled.set_custom_text(3, "Uptime: " + String(now / 1000) + " s");
        g_oled.show_custom();

        Serial.print("OLED refresh #");
        Serial.println(g_counter);
    }

    delay(10);
}
```

4. `OLED_t::begin()` 负责底层 I2C 初始化。程序先设置 SDA、SCL 引脚和总线时钟, 再尝试向 OLED 地址发送一次通信, 只有通信成功后才真正调用 `u8g2.begin()` 启动显示库。

```
bool OLED_t::begin()
{
    Wire.setPins(I2C_SDA_PIN, I2C_SCL_PIN);
    Wire.begin();
    Wire.setClock(400000);
    Wire.beginTransmission(OLED_I2C_ADDR);
    if (!Wire.endTransmission()) {
        u8g2.begin();
        return true;
    }
    return false;
}
```

5. `set_custom_text()` 只是把字符串写入 4 行缓存, 真正刷新屏幕是在 `show_custom()` 中完成的。
`show_custom()` 会按行设置光标位置, 并把缓存中的每一行文本依次显示出来。

```
void OLED_t::set_custom_text(uint8_t line, String text)
```

```

{
    if(line < 4) {
        custom_lines[line] = text;
    }
}

void OLED_t::show_custom()
{
    u8g2.firstPage();
    do {
        u8g2.setFont(u8g2_font_6x10_tr);
        for(int i = 0; i < 4; i++) {
            u8g2.setCursor(0, 15 + i * 15);
            u8g2.print(custom_lines[i]);
        }
    } while (u8g2.nextPage());
}

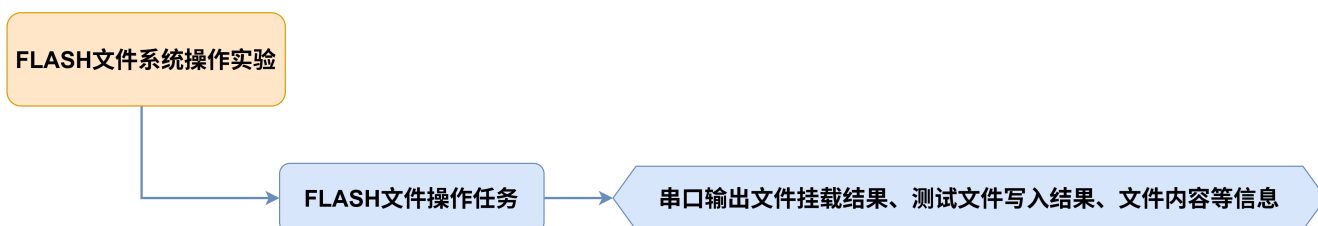
```

2.7 FLASH文件系统操作

2.7.1 项目介绍

本实验使用 ESP32 的 LittleFS 文件系统完成文件创建、写入、读取和目录遍历操作，学习 Flash 文件系统的基本使用方法。程序启动后会先挂载 LittleFS，然后创建测试文件，写入示例内容并立即读出，最后列出根目录中的文件信息。

2.7.2 实验流程



2.7.3 模块说明

ESP32 单片机的 **Flash 文件系统模块**是一种专为嵌入式设备设计的轻量级文件系统，运行于 ESP32 内置的 SPI Flash 存储空间之上。它允许开发者以文件读写的方式管理非易失性数据，无需依赖外部存储介质。该模块通过将 Flash 存储空间划分为若干逻辑分区，并在其上构建文件目录结构，使应用程序能够像操作普通文件系统一样创建、读取、写入和删除文件。由于 Flash 介质具有有限的擦写寿命，文件系统内部实现了磨损均衡算法，将写入操作均匀分散到各存储块，从而有效延长 Flash 的使用寿命。

2.7.4 程序下载

找到本文档同目录下的“02 程序源码”文件夹中的“07 FLASH文件操作\Flash_file”，打开“Flash_file.ino”，具体下载步骤可参考[Arduino程序下载](#)。

2.7.5 运行效果

程序运行后，串口会输出文件系统挂载结果、测试文件写入结果、文件内容以及根目录文件列表。之后每隔一段时间，串口会继续输出测试文件当前是否存在以及文件大小。

输出 串口监视器 x

消息 (按回车将消息发送到“COM3”上的“ESP32 Dev Module”)

```
>>> Cycle 2 (every 4 s)
[1] Create file: open /demo.txt for write, then close (empty file)
    OK: file exists, size = 0 bytes
[2] Write: open for append and write one line
    Written line: Nex_Arm LittleFS demo | cycle 2 | millis=16280
    OK: file size after write = 48 bytes
[3] Read: full file content
    Nex_Arm LittleFS demo | cycle 2 | millis=16280

-----

[4] Delete: remove /demo.txt from LittleFS
    OK: remove() succeeded.
    Verify: file is gone as expected.

-----

>>> Cycle 3 (every 4 s)
```

2.7.6 程序分析

1. 文件开头先定义测试文件路径、文件系统状态标志和定时上报间隔。后续无论是写文件、读文件还是定时检查，都是围绕 `/demo.txt` 这个测试文件展开的。

```
namespace {
constexpr const char* TEST_FILE = "/demo.txt";
bool g_fs_ready = false;
unsigned long g_last_report_ms = 0;
constexpr unsigned long REPORT_INTERVAL_MS = 3000;
}
```

2. `register_system_task()` 是整个文件系统实验的起点。程序先挂载 LittleFS，如果挂载失败就直接返回；挂载成功后，再按顺序执行写文件、读文件和遍历根目录三个操作。

```
void register_system_task(esp_event_loop_handle_t *event_loop)
{
    (void)event_loop;

    Serial.begin(115200);
    delay(100);

    g_fs_ready = LittleFS.begin(true);
    if (!g_fs_ready) {
        Serial.println("LittleFS mount failed.");
    }
}
```

```

        return;
    }

    Serial.println("FLASH(LittleFS) demo started.");
    write_demo_file();
    read_demo_file();
    list_root_files();
}

```

3. `write_demo_file()` 使用写模式打开 `/demo.txt`，如果文件不存在就会自动创建；随后写入两行测试文本，最后关闭文件并输出写入完成信息。

```

void write_demo_file()
{
    File file = LittleFS.open(TEST_FILE, "w");
    if (!file) {
        Serial.println("Failed to open demo file for write.");
        return;
    }

    file.println("Nex_Arm LittleFS demo");
    file.println("This file is created by system_task_handle.cpp");
    file.close();
    Serial.println("Demo file written.");
}

```

4. `read_demo_file()` 再次以只读方式打开同一个文件，并通过 `while (file.available())` 逐字节读取内容，最后把读取结果打印到串口中。

```

void read_demo_file()
{
    File file = LittleFS.open(TEST_FILE, "r");
    if (!file) {
        Serial.println("Failed to open demo file for read.");
        return;
    }

    Serial.println("Demo file content:");
    while (file.available()) {
        Serial.write(file.read());
    }
    Serial.println();
    file.close();
}

```

5. `list_root_files()` 通过 `openNextFile()` 依次遍历根目录中的文件，并输出文件名和大小。这样就可以直观看到刚才创建的测试文件已经被写入 Flash。

```

void list_root_files()
{
    File root = LittleFS.open("/");
}

```

```

    if (!root) {
        Serial.println("Failed to open LittleFS root.");
        return;
    }

    Serial.println("LittleFS root files:");
    File file = root.openNextFile();
    while (file) {
        Serial.print("  ");
        Serial.print(file.name());
        Serial.print(" (");
        Serial.print(file.size());
        Serial.println(" bytes)");
        file = root.openNextFile();
    }
}

```

6. 主循环中的 `system_loop_handler()` 负责周期性检查文件是否仍然存在。只有在文件系统已经挂载成功的前提下，程序才会每隔 3 秒重新打开一次测试文件并输出当前大小。

```

void system_loop_handler(void)
{
    if (!g_fs_ready) {
        delay(100);
        return;
    }

    unsigned long now = millis();
    if (now - g_last_report_ms >= REPORT_INTERVAL_MS) {
        g_last_report_ms = now;

        File file = LittleFS.open(TEST_FILE, "r");
        if (file) {
            Serial.print("Demo file still exists, size = ");
            Serial.print(file.size());
            Serial.println(" bytes");
            file.close();
        } else {
            Serial.println("Demo file not found.");
        }
    }

    delay(10);
}

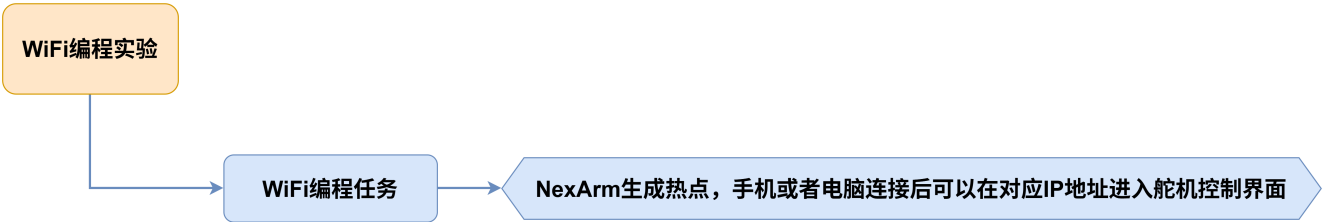
```

2.8 WIFI编程操作

2.8.1 项目介绍

本实验让 ESP32 工作在 AP 模式，创建一个热点并启动网页服务器，通过浏览器控制 6 个总线舵机的目标位置。程序会将当前舵机位置缓存为数组，通过网页滑块发送控制请求，再调用舵机驱动接口完成动作执行。

2.8.2 实验流程



2.8.3 模块说明

ESP32 单片机的 **Wi-Fi 模块**是其核心无线通信功能之一，内置符合 IEEE 802.11 b/g/n 标准的 Wi-Fi 控制器，支持 2.4GHz 频段，最高数据传输速率可达 150Mbps。该模块支持 Station（STA）、Access Point（AP）以及 Station+AP 共存三种工作模式：在 STA 模式下，ESP32 作为客户端接入现有无线网络；在 AP 模式下，ESP32 自身作为热点供其他设备连接；在共存模式下则可同时承担两种角色，灵活适应复杂的网络拓扑需求。Wi-Fi 模块内置完整的 TCP/IP 协议栈，支持 DHCP、DNS、HTTP、HTTPS、MQTT 等常用网络协议，开发者可借助 ESP-IDF 或 Arduino 框架提供的高层 API 快速实现网络连接、数据收发及云平台对接等功能。

2.8.4 程序下载

找到本文档同目录下的“02 程序源码”文件夹中的“08 WIFI编程操作\WiFi_Test”，打开“WiFi_Test.ino”，具体下载步骤可参考[Arduino程序下载](#)。

2.8.5 运行效果

程序运行后，串口会输出热点名称、密码和访问地址。

```
输出  串口监视器 x
消息 (按回车将消息发送到“COM3”上的“ESP32 Dev Module”)

SSID: NexArm_Servo_Control
Password: 12345678
Open in browser: http://192.168.4.1
WiFi servo control demo started.
SSID: NexArm_Servo_Control
Password: 12345678
Open in browser: http://192.168.4.1
WiFi servo control demo started.
SSID: NexArm_Servo_Control
Password: 12345678
Open in browser: http://192.168.4.1
WiFi servo control demo started.
SSID: NexArm_Servo_Control
Password: 12345678
Open in browser: http://192.168.4.1
```

使用手机或电脑连接该热点，在浏览器中打开对应 IP 地址"<http://192.168.4.1>"后，可以看到 6 路舵机控制页面；拖动滑块时，舵机会移动到指定目标位置。



2.8.6 程序分析

1. 主程序开头先定义热点名称、密码、舵机数量、默认位置以及网页服务器对象。`g_servo_positions` 数组用于缓存 6 路舵机当前的目标位置，网页读取和舵机控制都围绕这个数组展开。

```
namespace {
constexpr const char* AP_SSID = "NexArm_Servo_Control";
constexpr const char* AP_PASS = "12345678";
constexpr uint16_t SERVO_MIN_POS = 0;
constexpr uint16_t SERVO_MAX_POS = 4096;
constexpr uint8_t SERVO_COUNT = 6;
constexpr int16_t SERVO_SPEED = 0;
constexpr uint16_t SERVO_DEFAULT_POS = 2048;

webServer g_server(80);
uint16_t g_servo_positions[SERVO_COUNT] = {
    SERVO_DEFAULT_POS,
    SERVO_DEFAULT_POS,
    SERVO_DEFAULT_POS,
    SERVO_DEFAULT_POS,
    SERVO_DEFAULT_POS,
    SERVO_DEFAULT_POS
};
}
```

2. 网页界面直接写在 `INDEX_HTML` 字符串中。页面加载后会动态生成 6 个滑块，并在滑块变化时通过 `fetch('/set?...')` 向 ESP32 发送控制请求，因此浏览器本身就是这个实验的上位机界面。

```
const char INDEX_HTML[] PROGMEM = R"HTML(
<script>
    const SERVO_COUNT = 6;
    const grid = document.getElementById('servoGrid');
    const timers = new Array(SERVO_COUNT).fill(null);

    for (let i = 1; i <= SERVO_COUNT; i++) {
        const card = document.createElement('div');
```

```

    card.className = 'card';
    card.innerHTML = `
      <div class="servo-title">Servo ${i}</div>
      <input class="slider" id="slider-${i}" type="range" min="0" max="4096" step="1"
value="2048">
      <div class="current-label">Current Target</div>
      <div class="current-value" id="value-${i}">2048</div>
    `;
    grid.appendChild(card);

    const slider = document.getElementById(`slider-${i}`);
    const label = document.getElementById(`value-${i}`);
    slider.addEventListener('input', () => {
      label.textContent = slider.value;
      clearTimeout(timers[i - 1]);
      timers[i - 1] = setTimeout(() => {
        fetch(`/set?id=${i}&pos=${slider.value}`).catch(() => {});
      }, 40);
    });
  }
</script>
)HTML";

```

3. `register_system_task()` 负责硬件和网络初始化。程序先打开舵机串口，再把 ESP32 设置为 AP 模式，随后配置固定 IP、创建热点、注册 HTTP 路由并启动 WebServer。

```

void register_system_task(esp_event_loop_handle_t *event_loop)
{
  (void)event_loop;

  Serial.begin(115200);
  delay(100);

  servo.begin(Serial1, 1000000, 16, 17);

  WiFi.mode(WIFI_AP);
  IPAddress local_ip(192, 168, 4, 1);
  IPAddress gateway(192, 168, 4, 1);
  IPAddress subnet(255, 255, 255, 0);
  WiFi.softAPConfig(local_ip, gateway, subnet);

  if (WiFi.softAP(AP_SSID, AP_PASS)) {
    Serial.println("WiFi servo control demo started.");
    Serial.print("SSID: ");
    Serial.println(AP_SSID);
    Serial.print("Password: ");
    Serial.println(AP_PASS);
    Serial.print("Open in browser: http://");
    Serial.println(WiFi.softAPIP());
  } else {
    Serial.println("Failed to start WiFi AP.");
  }
}

```

```

g_server.on("/", HTTP_GET, handleRoot);
g_server.on("/positions", HTTP_GET, handlePositions);
g_server.on("/set", HTTP_GET, handleSet);
g_server.begin();
}

```

4. 页面与控制器之间通过 HTTP 接口交换数据。主页路由返回完整网页，`/positions` 路由则返回 JSON 数据，让浏览器能够读取当前缓存的舵机目标位置。

```

String buildPositionsJson()
{
    String json = "{\"p\":[";
    for (uint8_t i = 0; i < SERVO_COUNT; ++i) {
        if (i > 0) {
            json += ",";
        }
        json += String(g_servo_positions[i]);
    }
    json += "]}";
    return json;
}

void handleRoot()
{
    g_server.send_P(200, "text/html; charset=utf-8", INDEX_HTML);
}

void handlePositions()
{
    g_server.send(200, "application/json", buildPositionsJson());
}

```

5. `handleSet()` 负责真正的控制逻辑。函数先检查参数是否齐全，再检查舵机编号是否越界，然后对目标位置做范围限制，最后调用 `servo.write_pos_ex()` 把位置写给指定舵机。

```

void handleSet()
{
    if (!g_server.hasArg("id") || !g_server.hasArg("pos")) {
        g_server.send(400, "application/json", "{\"ok\":false,\"msg\":\"missing args\"}");
        return;
    }

    int id = g_server.arg("id").toInt();
    int pos = g_server.arg("pos").toInt();

    if (id < 1 || id > SERVO_COUNT) {
        g_server.send(400, "application/json", "{\"ok\":false,\"msg\":\"bad id\"}");
        return;
    }

    pos = constrain(pos, SERVO_MIN_POS, SERVO_MAX_POS);
}

```

```
g_servo_positions[id - 1] = static_cast<uint16_t>(pos);  
servo.write_pos_ex(static_cast<uint8_t>(id), 0, SERVO_SPEED, static_cast<int16_t>(pos));  
  
g_server.send(200, "application/json", "{\"ok\":true}");  
}
```

6. 服务器启动后，还需要在主循环中不断调用 `handleClient()` 才能处理浏览器请求。也就是说，网页控制能否及时响应，取决于 `system_loop_handler()` 是否持续运行。

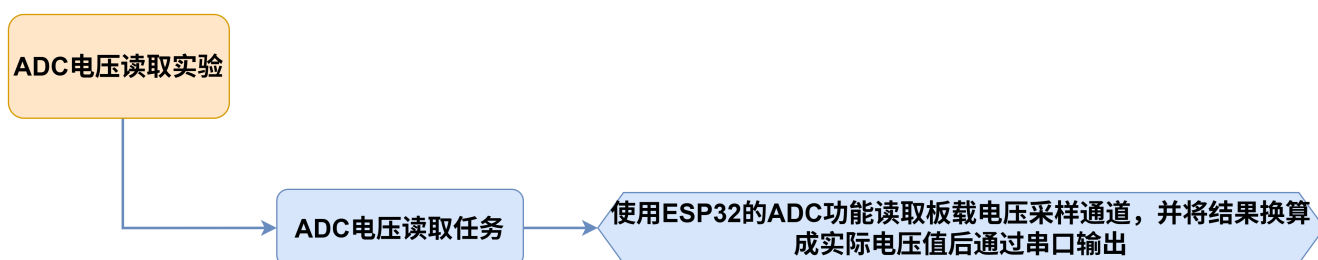
```
void system_loop_handler(void)  
{  
    g_server.handleClient();  
    delay(2);  
}
```

2.9 ADC电压读取

2.9.1 项目介绍

本实验使用 ESP32 的 ADC 功能读取板载电压采样通道，并将结果换算成实际电压值后通过串口输出。程序中加入了简单的滑动平均滤波，能够让电压读数更加稳定，便于观察电池电压变化。

2.9.2 实验流程



2.9.3 模块说明

ESP32 单片机的 **ADC（模数转换器）模块**是其重要的模拟信号采集功能单元，内置两个独立的 SAR（逐次逼近型）ADC 控制器，分别为 ADC1 和 ADC2，共支持多达 18 个模拟输入通道，分布于多个 GPIO 引脚之上。该模块支持 9 位、10 位、11 位及 12 位四种分辨率可配置模式，在 12 位分辨率下可将 0~3.3V 的模拟输入信号转换为 0~4095 的数字量，满足不同精度需求的应用场景。ADC 模块内置可编程衰减器，支持 0dB、2.5dB、6dB 和 11dB 四档增益衰减设置，通过调整衰减系数可灵活扩展有效测量电压范围，最高可测量接近 3.9V 的输入信号。

2.9.4 程序下载

找到本文档同目录下的“02 程序源码”文件夹中的“09 adc电压读取\ADC_Read”，打开“ADC_Read.ino”，具体下载步骤可参考[Arduino程序下载](#)。

2.9.5 运行效果

程序运行后，串口会每秒输出一次当前电压值，既包含毫伏数值，也包含换算后的伏特值，便于直接观察供电电压变化情况。

```
输出  串口监视器 x
消息 (按回车将消息发送到"COM3"上的"ESP32 Dev Module")

>>> Cycle 2 (every 4 s)
[1] Create file: open /demo.txt for write, then close (empty file)
    OK: file exists, size = 0 bytes
[2] Write: open for append and write one line
    Written line: Nex_Arm LittleFS demo | cycle 2 | millis=16280
    OK: file size after write = 48 bytes
[3] Read: full file content
    Nex_Arm LittleFS demo | cycle 2 | millis=16280

[4] Delete: remove /demo.txt from LittleFS
    OK: remove() succeeded.
    Verify: file is gone as expected.
-----

>>> Cycle 3 (every 4 s)
```

2.9.6 程序分析

1. 主程序先在匿名命名空间中定义电压采样对象和打印周期。后面的主循环只需要对照 `PRINT_INTERVAL_MS` 判断是否到达下一次输出时间即可。

```
namespace {
Bat_t g_bat;
unsigned long g_last_print_ms = 0;

constexpr unsigned long PRINT_INTERVAL_MS = 1000;
}
```

2. `register_system_task()` 负责初始化串口和电压采样模块，`system_loop_handler()` 则持续调用 `g_bat.update()` 更新最新采样值，并每秒输出一一次计算结果。

```
void register_system_task(esp_event_loop_handle_t *event_loop)
{
    (void)event_loop;

    Serial.begin(115200);
    delay(100);

    g_bat.begin();

    Serial.println("ADC voltage demo started.");
}
```

```

        Serial.println("Reading battery voltage every 1 second.");
    }

    void system_loop_handler(void)
    {
        unsigned long now = millis();
        g_bat.update();

        if (now - g_last_print_ms >= PRINT_INTERVAL_MS) {
            g_last_print_ms = now;

            int voltage_mv = g_bat.get_voltage();
            Serial.print("Battery voltage: ");
            Serial.print(voltage_mv);
            Serial.print(" mV (");
            Serial.print(voltage_mv / 1000.0f, 2);
            Serial.println(" V)");
        }

        delay(10);
    }
}

```

3. `Bat_t` 类在头文件中定义了滤波窗口大小、缓存区和校准结构体。这里的 `filter_buf` 用来保存最近若干次采样结果，后续程序会对这些数据求平均值。

```

#define WINDOWS_SIZE 10

class Bat_t {
public:
    void begin(void);
    void update(void);
    int get_voltage(void);

private:
    uint8_t filter_index = 0;
    int voltage;
    int filter_buf[WINDOWS_SIZE] = {0};
    esp_adc_cal_characteristics_t adc_chars;
};

```

4. `Bat_t::begin()` 中先配置 ADC 通道衰减参数，再调用 `esp_adc_cal_characterize()` 建立校准信息，最后先执行一次 `update()`，这样系统启动后就能得到首个电压值。

```

void Bat_t::begin()
{
    adc1_config_channel_atten(ADC_PIN, ADC_ATTEN);
    esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN, ADC_WIDTH, DEFAULT_VREF, &adc_chars);
    update();
}

```

5. `Bat_t::update()` 是采样的核心函数。程序先读取 ADC 原始值，再换算为毫伏值；由于板上使用了分压电路，因此又乘以 `11.0f` 恢复实际输入电压，最后写入滤波缓存并计算平均值。

```
void Bat_t::update()
{
    int raw = adc1_get_raw(ADC_PIN);
    int samples_voltage = esp_adc_cal_raw_to_voltage(raw, &adc_chars);
    samples_voltage = (int)(11.0f * (float)samples_voltage);
    filter_buf[filter_index] = samples_voltage;
    filter_index = (filter_index + 1) % WINDOWS_SIZE;

    int sum = 0;
    if(filter_buf[WINDOWS_SIZE - 1] != 0) {
        for (uint8_t i = 0; i < WINDOWS_SIZE; i++) sum += filter_buf[i];
        voltage = sum / WINDOWS_SIZE;
    }
}
```

6. `get_voltage()` 只是把已经滤波完成的电压结果返回给上层调用，主循环拿到这个结果后，再把它同时格式化成毫伏和伏特两种形式输出。

```
int Bat_t::get_voltage()
{
    return voltage;
}
```